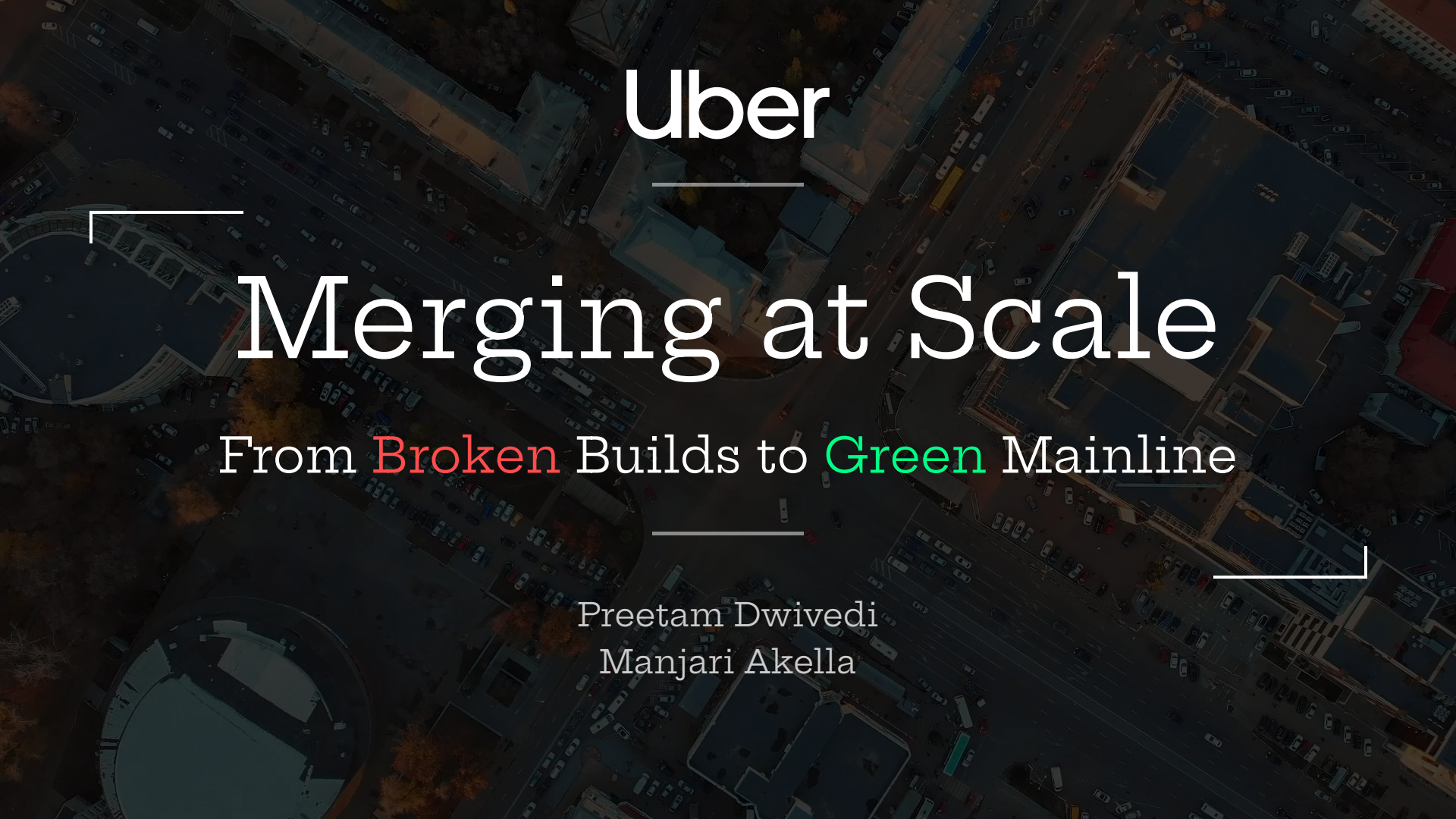




Uber

Merging at Scale

From **Broken** Builds to **Green** Mainline

Preetam Dwivedi
Manjari Akella

About Us



Preetam Dwivedi
Staff Engineer,
Developer Platform,
Uber



Manjari Akella
Sr. Engineer,
Developer Platform,
Uber

Outline

- Problem
- Approach
- Core Concepts
- Architecture
- Evolution & Results
- Limitations & Future Plan

Monorepos

One Codebase, Many Tradeoffs

One or Many?



Monorepo

OR



Poly Repo

Monorepos at Uber

DEVELOPMENT SCALE

5K+ Contributors

36K+ Commits / Wk

27K PRs / Wk

440M+ Lines of Code

MONOREPO STATS BY LANGUAGE



35 GB

227M+ LOC
22K commits/wk



Web

18 GB

81M LOC
4K commits/wk



Java

17 GB

57M+ LOC
3.4K commits/wk



Python

11 GB

40M LOC
3.2K commits/wk

iOS

9 GB

20M LOC
1.2K commits/wk

Android

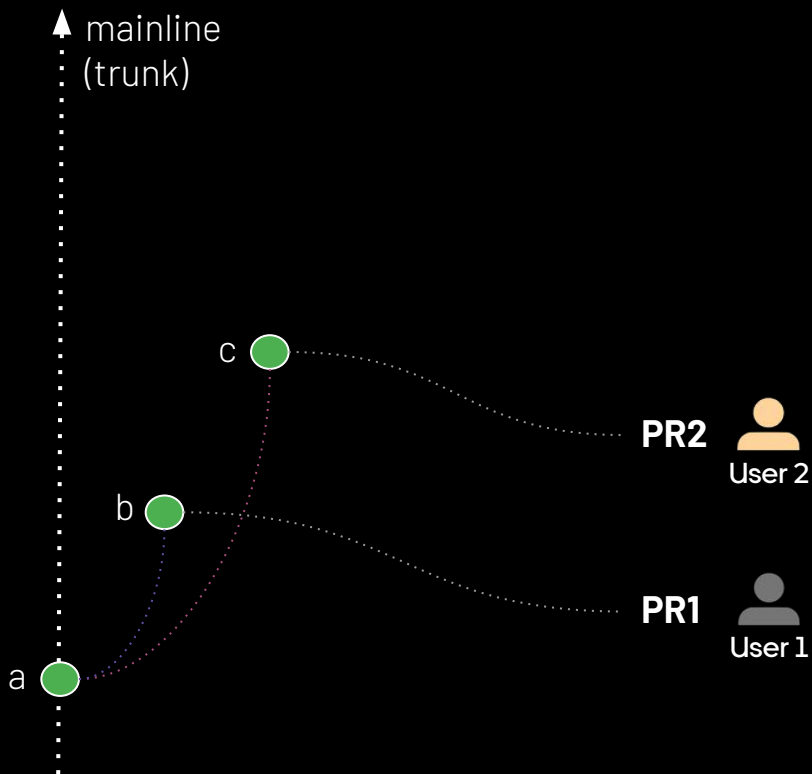
7 GB

16M LOC
1.8K commits/wk

Problem Statement

Green PRs don't guarantee a green mainline

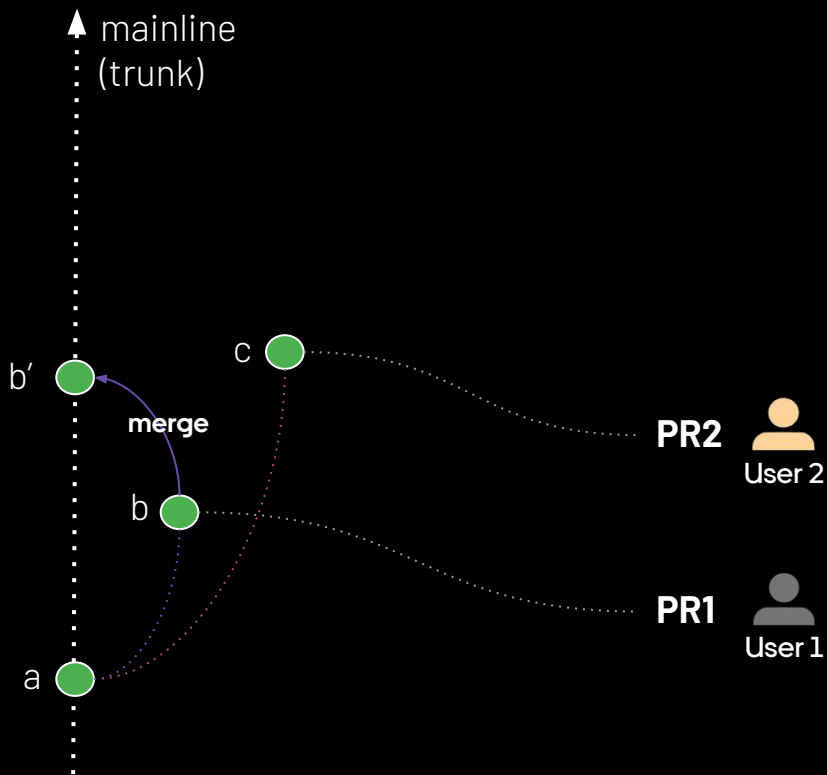
Green Builds on Pull Requests



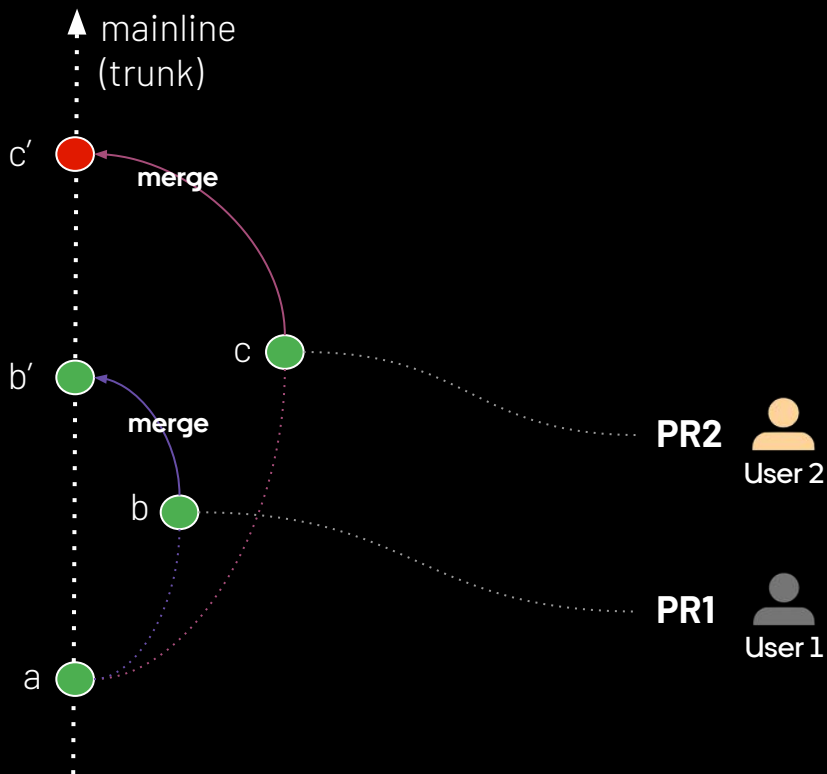
Isolated Success

- PRs are tested against base **a**
- CI confirms **b** and **c** are valid
- Both authors see green builds

Broken Builds on Mainline



Broken Builds on Mainline



! Integration Failure

- **State Drift:** PR2 (c) was tested against base a, but merged after PR1 (b').
- **False Confidence:** PR2 passed all tests in isolation.
- **Result:** Mainline is now broken for every other engineer.

Imagine Thousands of Engineers

At scale, small problems don't stay small



The Blast Radius

A single broken merge doesn't just fail; it blocks thousands of engineers.



Org-Wide Burden

Debugging becomes a shared, collective burden across the entire organization.



Productivity Loss

Productivity stalls as integration friction outweighs individual progress.

How do we keep mainline green at scale?

SubmitQueue

Think of this as an intelligent merge queue that ensures mainline stays green

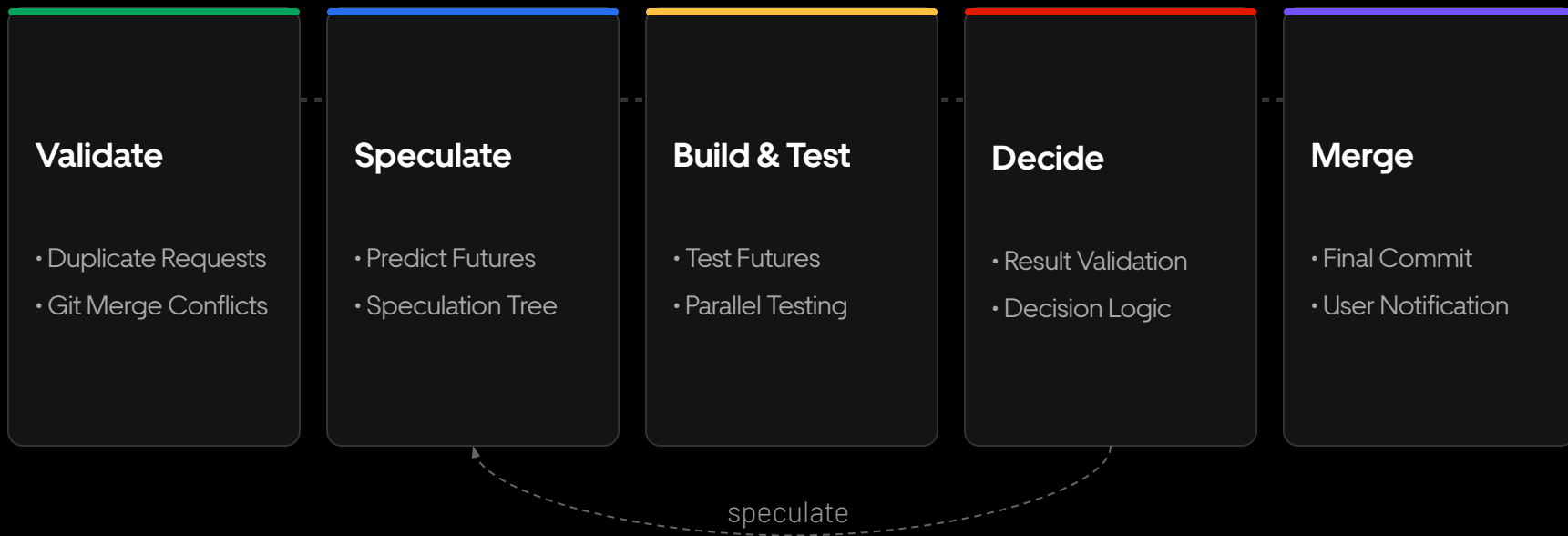
SubmitQueue: Uber's Merge Queue

- A speculative merge queue that validates future mainline states before merging
- Ensures a consistently green mainline without sacrificing velocity

This isn't just a queue — it's a system that predicts the future of your mainline before changes merge.

Request Journey

Every request is tested against future mainline states before it lands (aka merges)

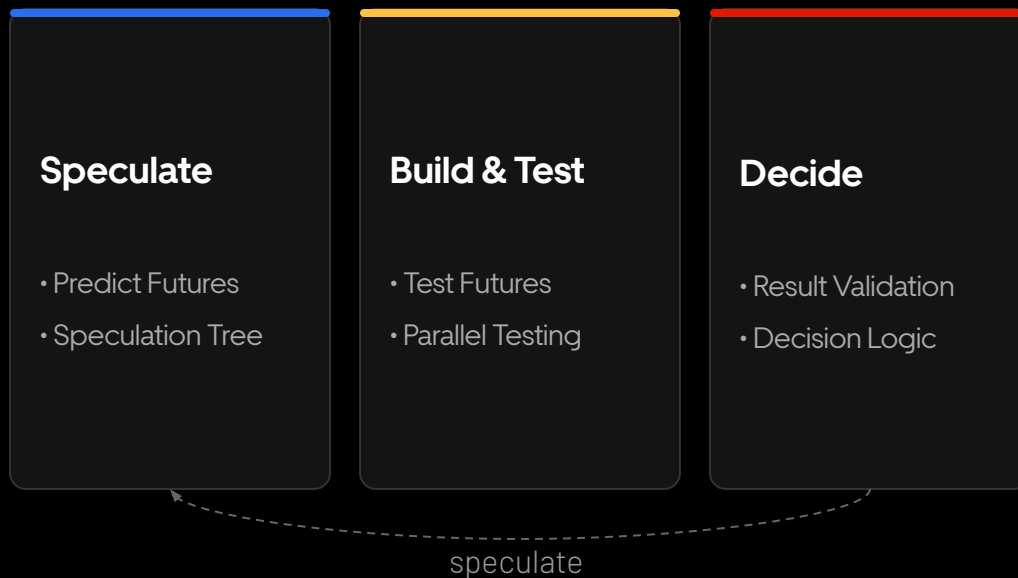


The Messy Middle

Core concepts behind large-scale merging

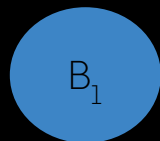
Speculation

Predicting and validating future states before they happen



How this works: An Example

Speculation Tree (C1)

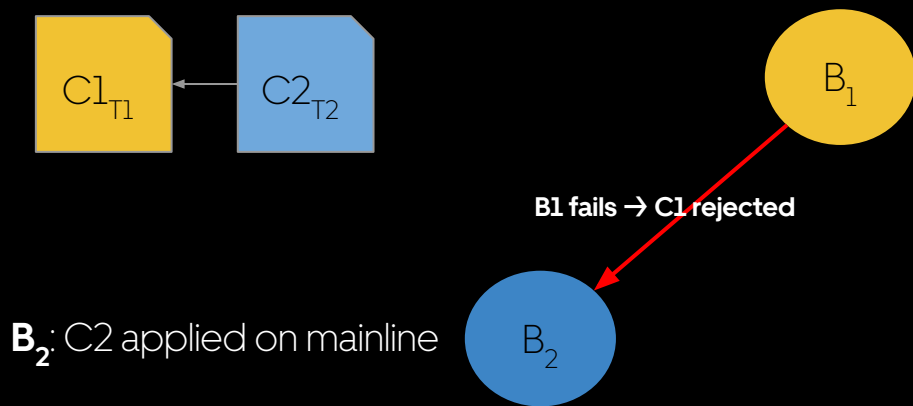


B_1 : C1 applied on mainline

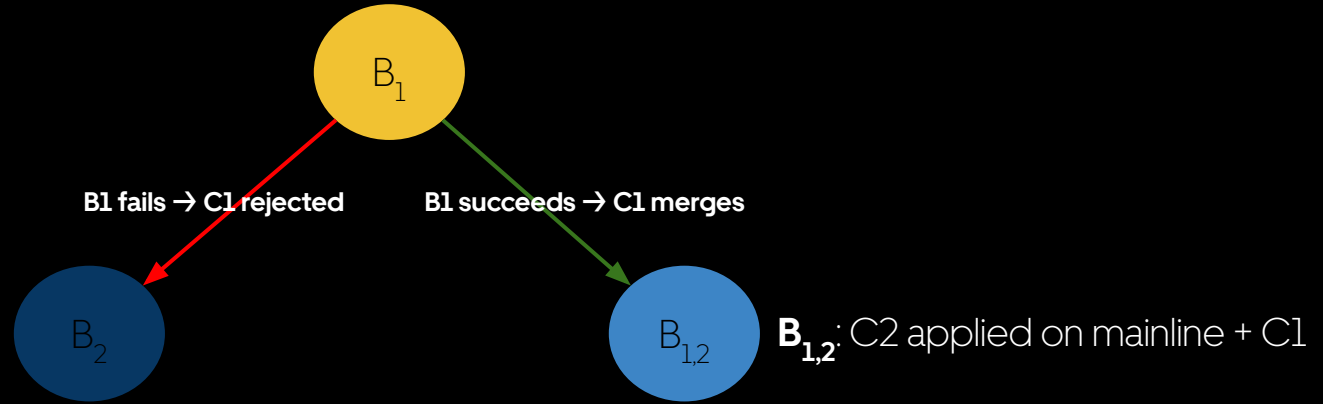
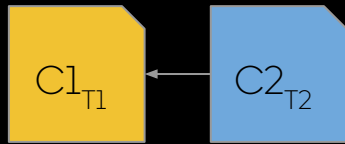
Assumption:

Each new change is assumed to depend on all earlier changes still in the queue.

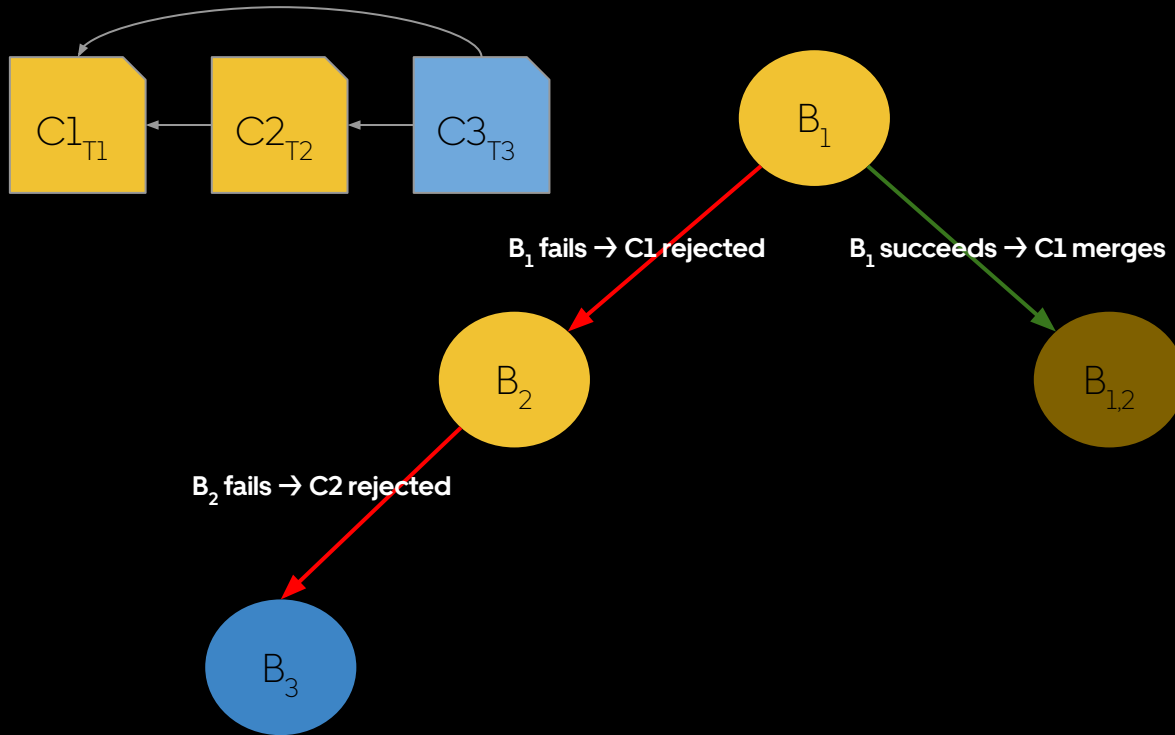
Speculation Tree (C2)



Speculation Tree (C2)

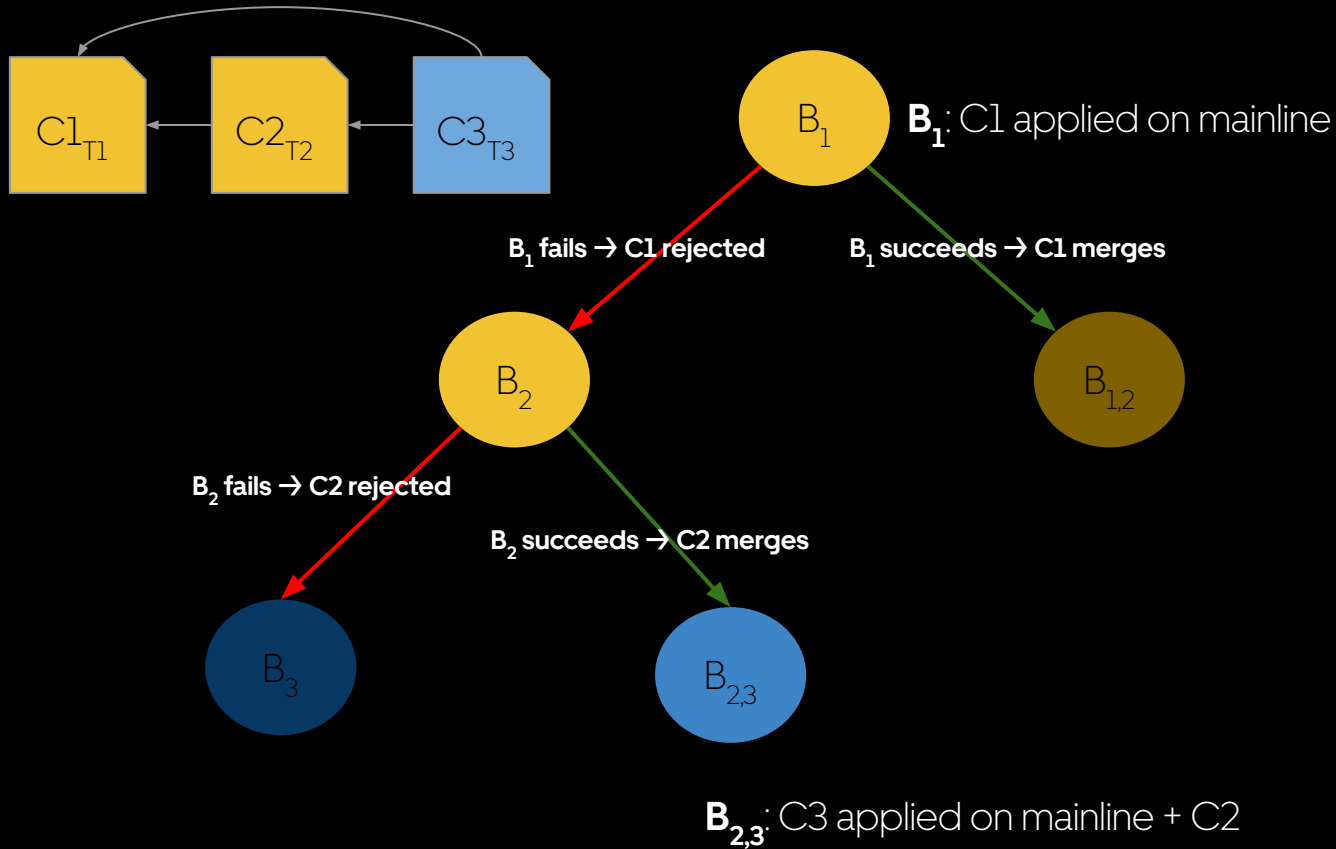


Speculation Tree (C3)

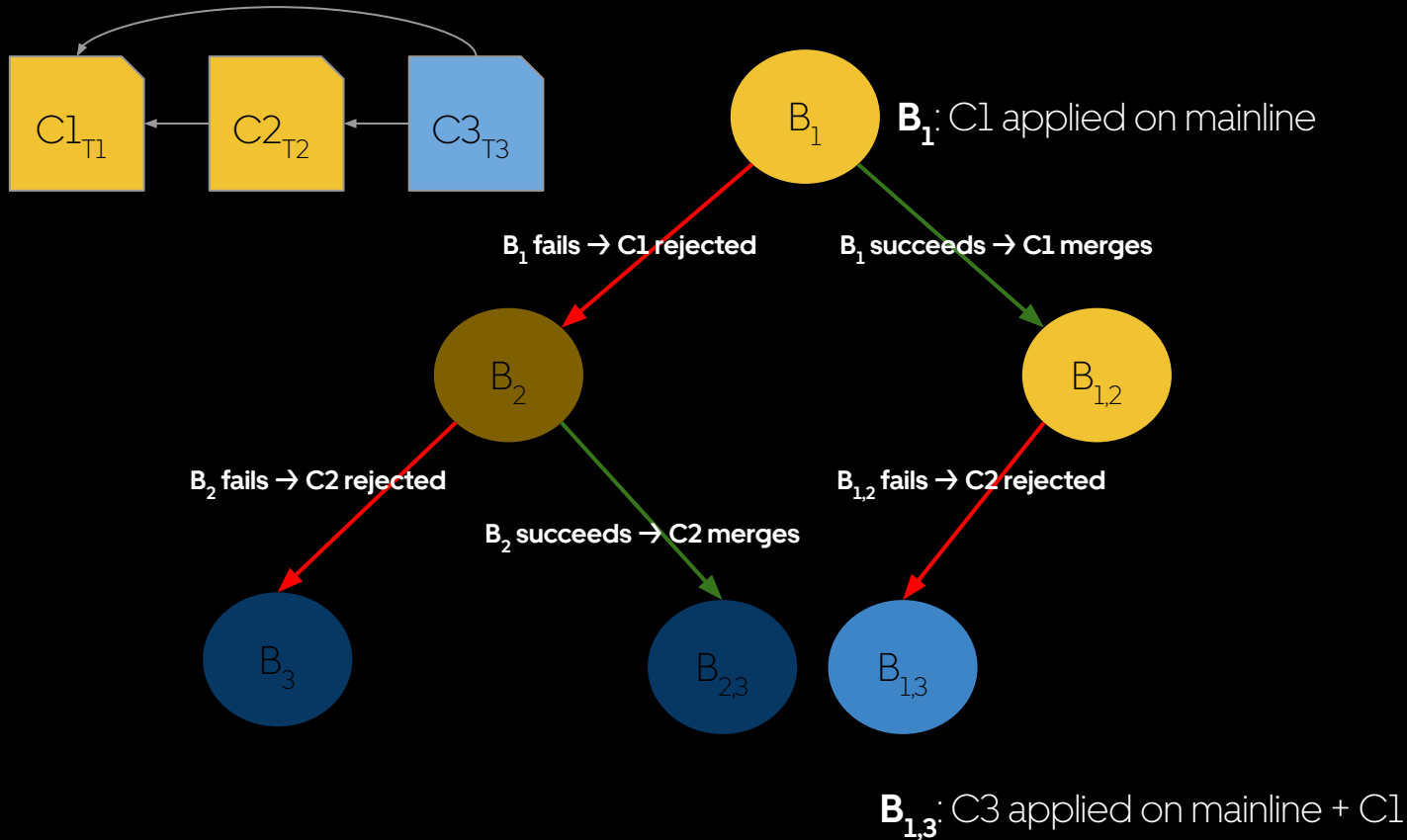


B_3 : C3 applied on mainline

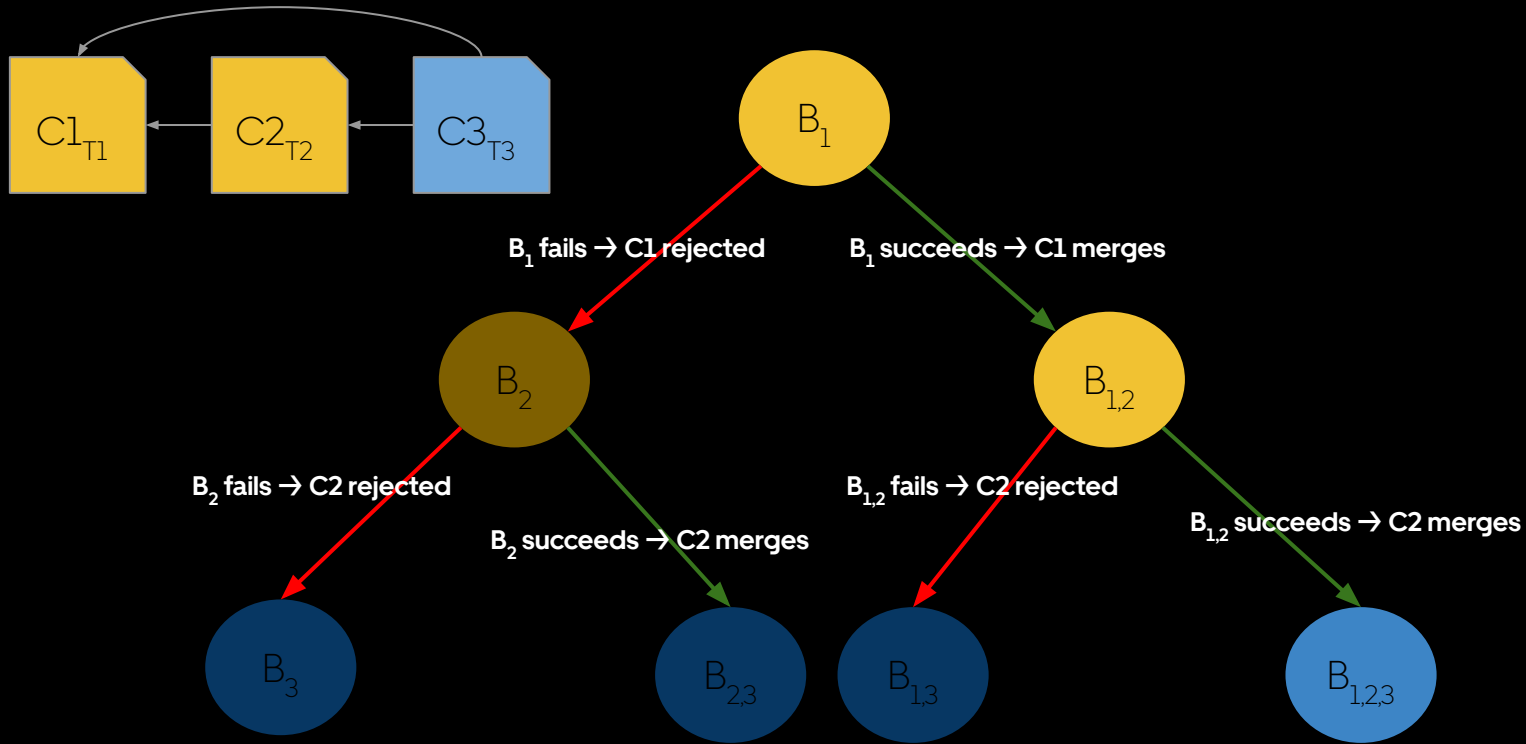
Speculation Tree (C3)



Speculation Tree (C3)

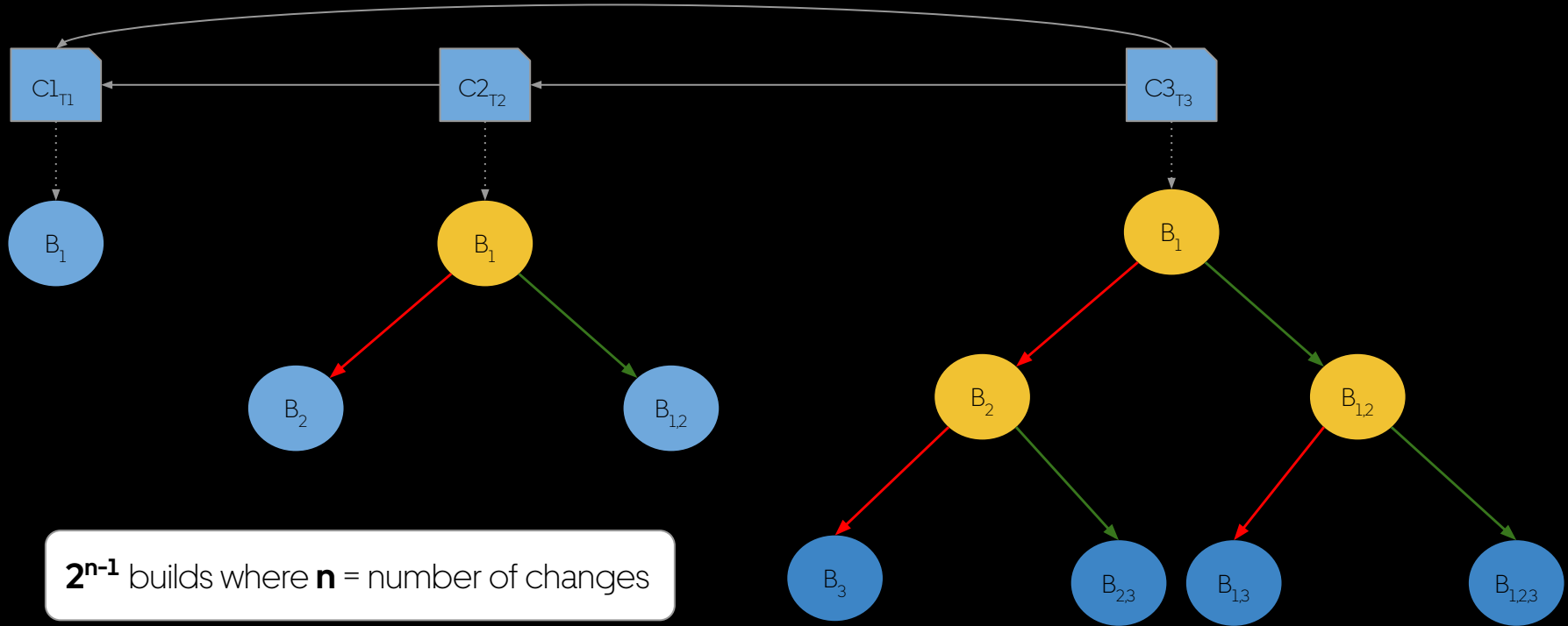


Speculation Tree (C3)



$B_{1,2,3}$: $C3$ applied on mainline + $C1$ + $C2$

Speculation Trees



Scale Challenge: Exponential Build Growth

Speculative testing hits a wall as codebase complexity increases



Complexity Wall

Testing every state requires $O(2^n)$ builds, making it **operationally infeasible** at Uber's scale.



Resource Waste

CI resources are limited and expensive. Most speculative paths are discarded, wasting massive compute.

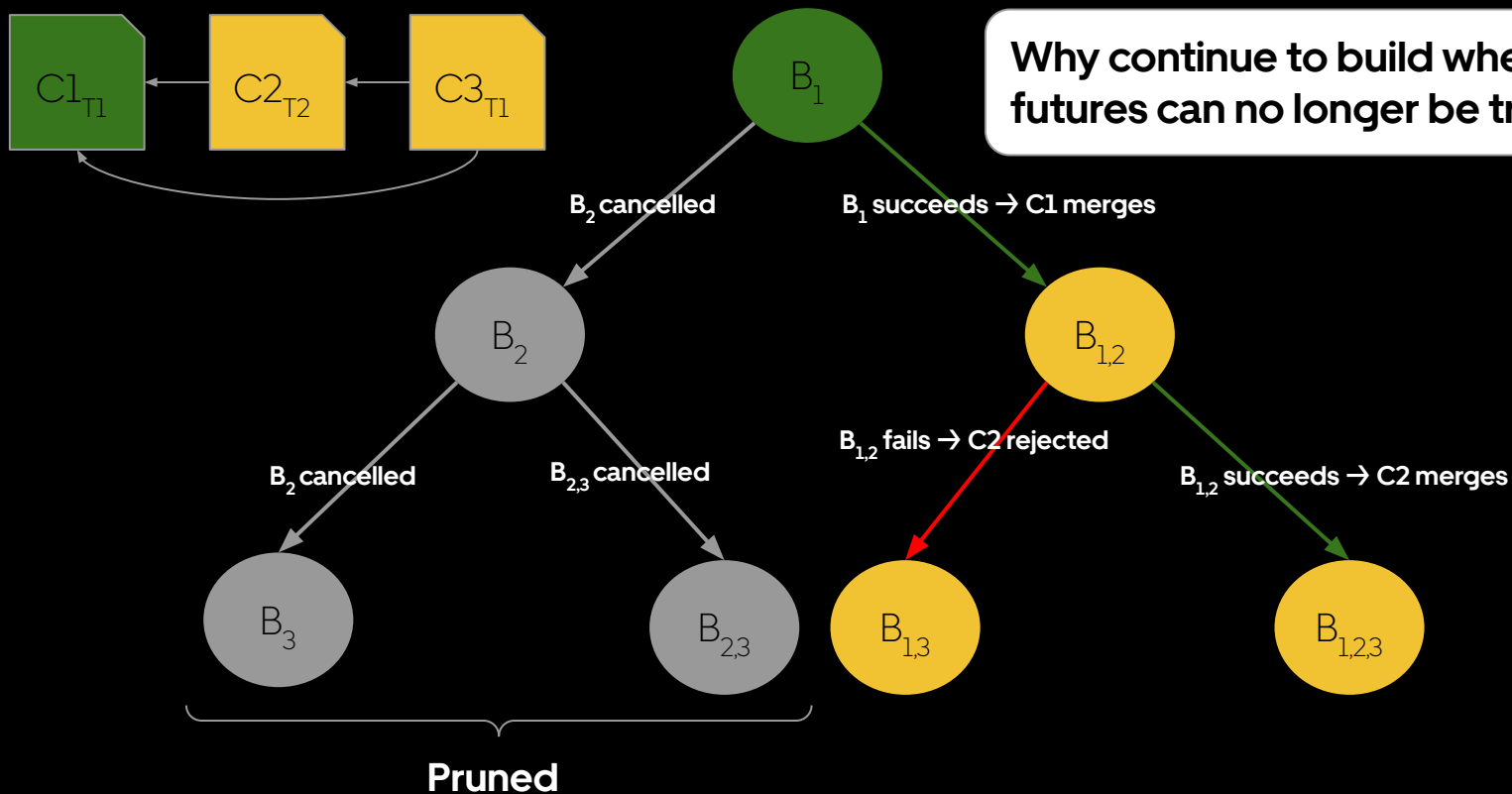


Velocity Drag

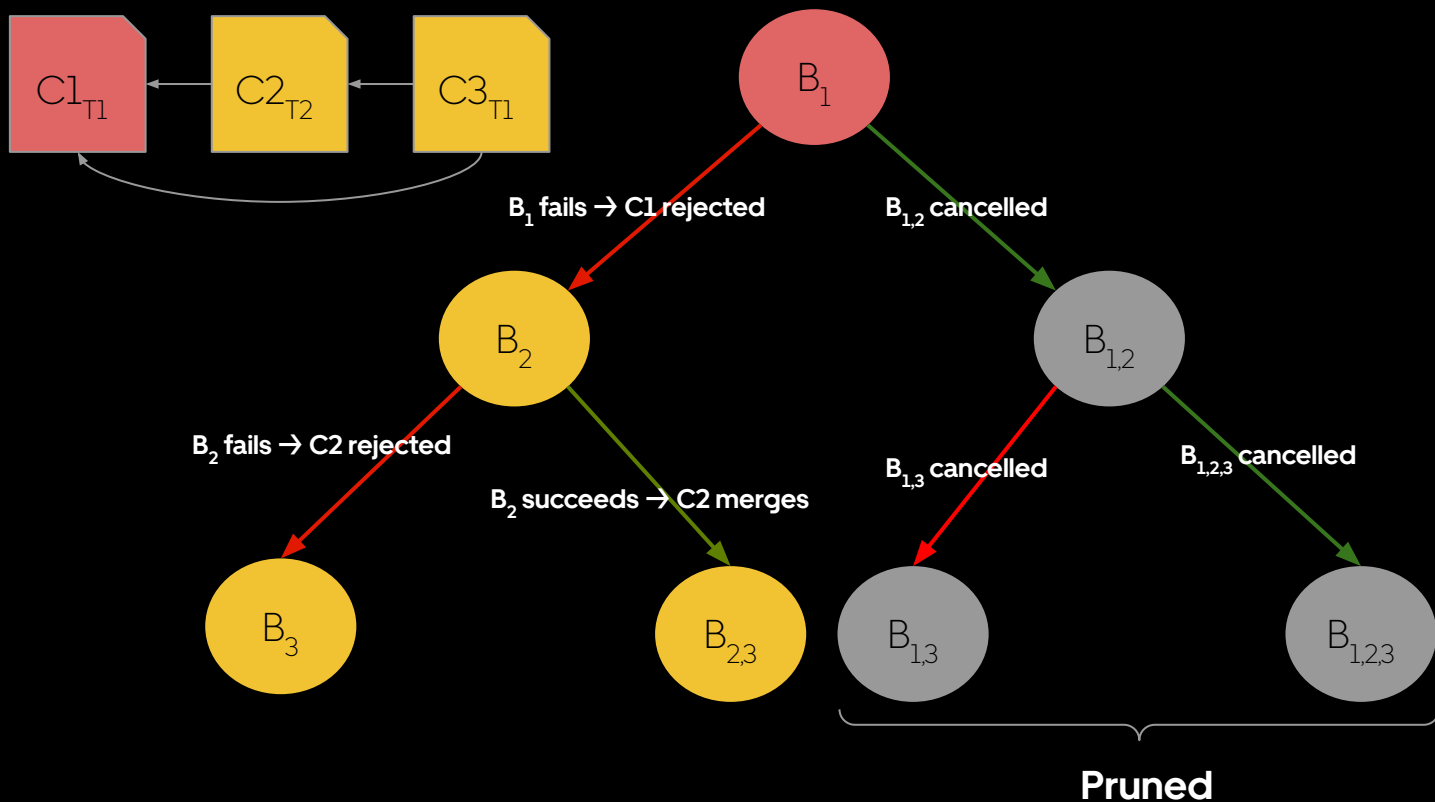
Current methods are **too slow** for large codebases with continuous, high-frequency changes.

How do we solve this problem?

Optimization-1: Pruning



Optimization-1: Pruning



Optimization-2: Dependency Mapping

Why assume every change depends on one another?



Change Identification

Isolate exactly which components are affected.



Overlap Detection

Detect and manage simultaneous changes.



Build Optimization

Avoid unnecessary work to save CI resources.

Let's understand with examples

Optimization-2: Dependency Mapping

REPO

src/org/group/service1/...

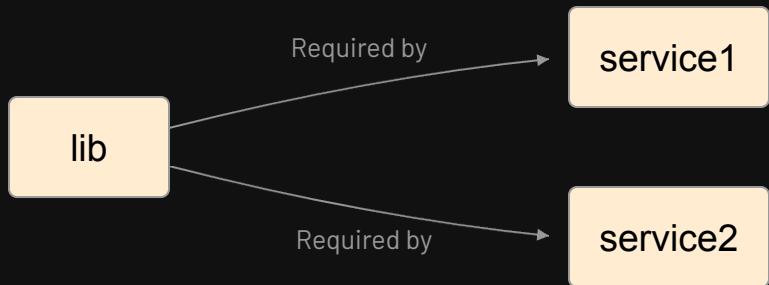
src/org/group/service2/...

src/org/group/common/lib/...

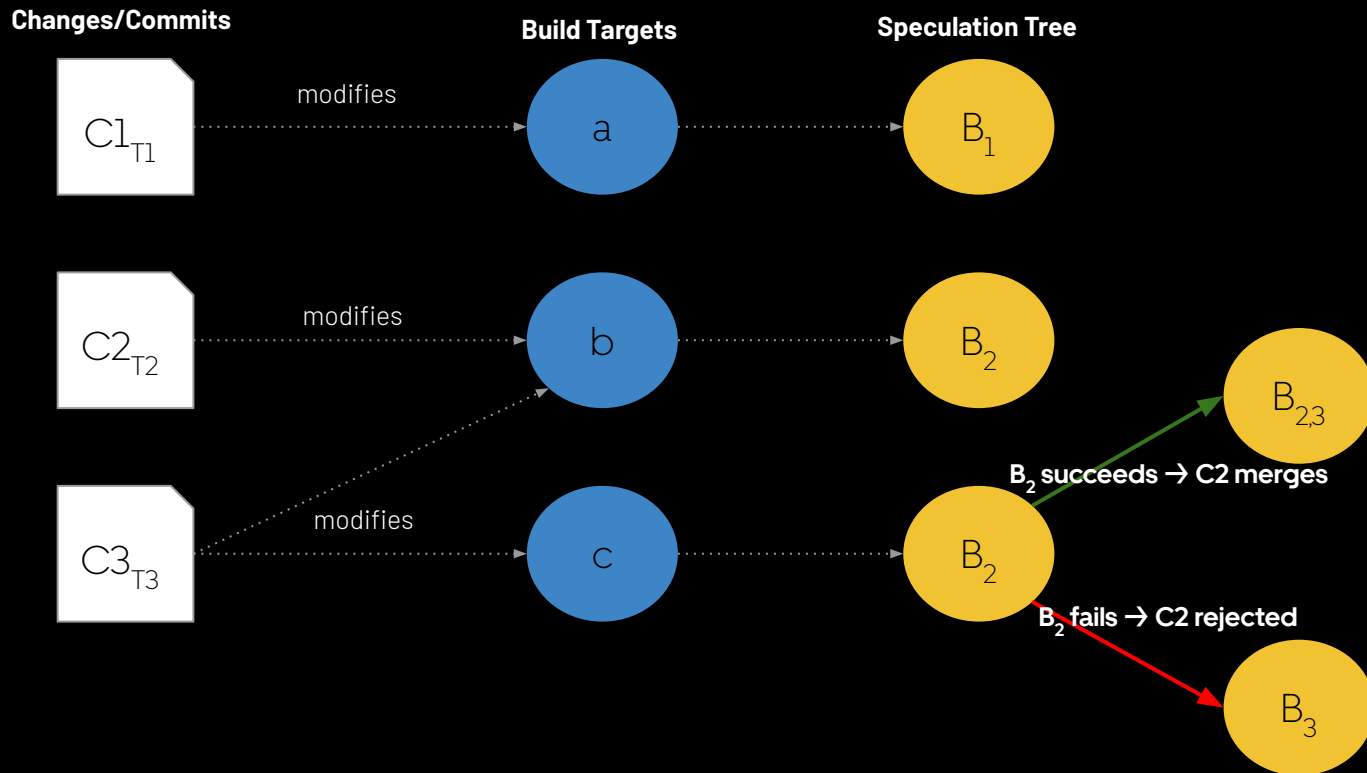
...

Assumption $\neq$ Every change depends on each other

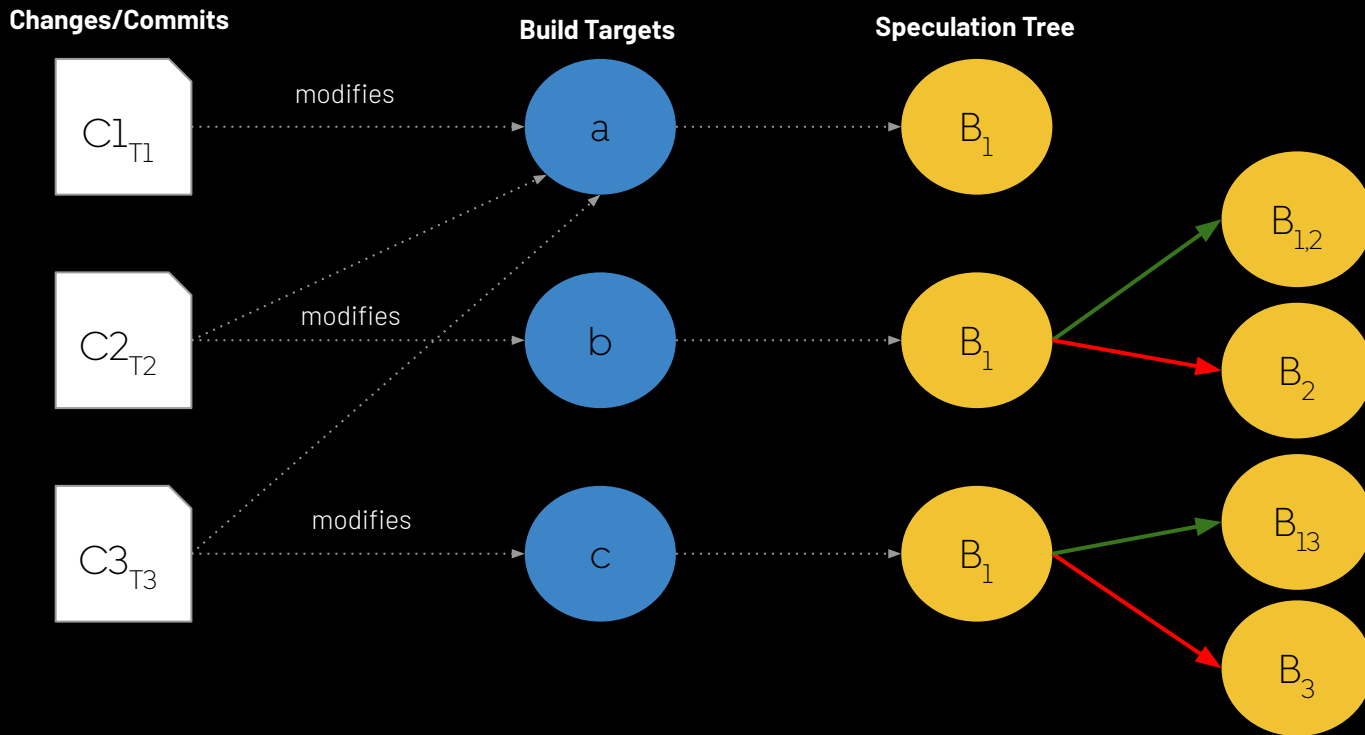
BUILD TARGET DEPENDENCY GRAPH



Optimization-2: Dependency Mapping



Optimization-2: Dependency Mapping



Optimization-3: Probabilistic Speculation

Bet CI resources on builds most likely to succeed



Predict Probability

Quantify success likelihood for every potential build state.



Prioritize Outcomes

Focus resources on branches most likely to reach the mainline.



Reduce Wasted Builds

Spawn builds for futures that are most needed.

Let's look at an example to understand?

Probabilistic Speculation Tree (C1)

$s(C1)$

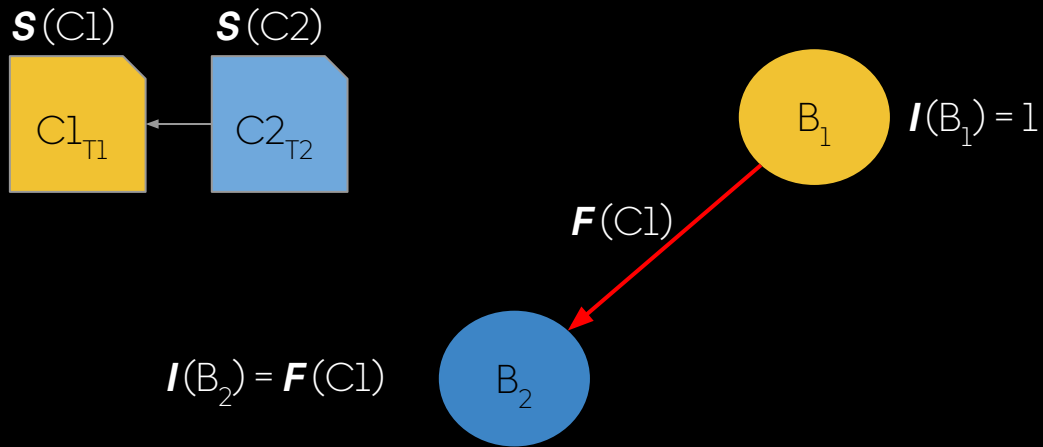


$$\mathbf{Success}(C1) = 1 - \mathbf{Failure}(C1)$$

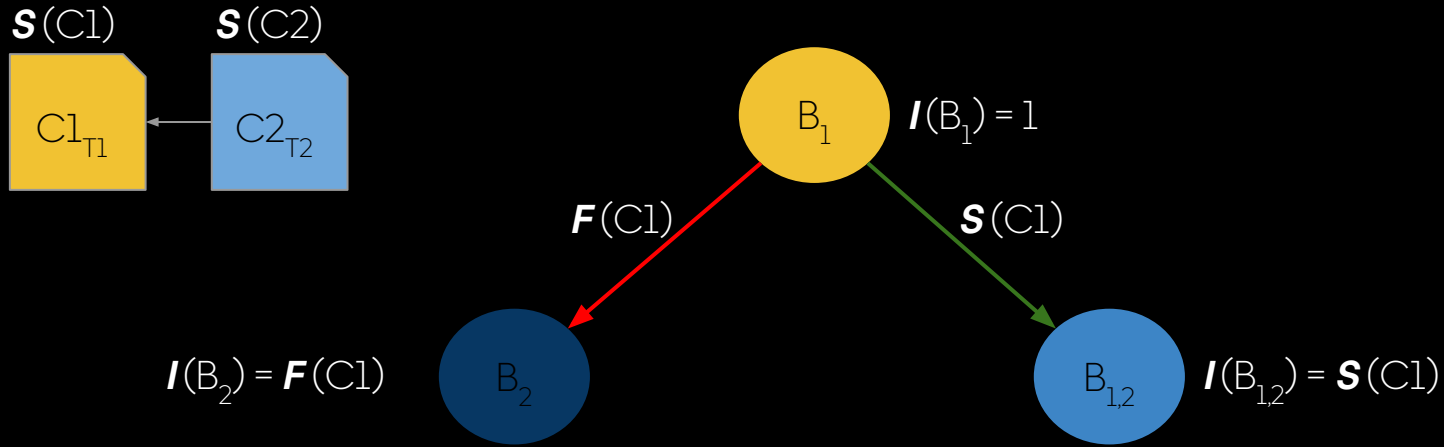
$$\mathbf{Failure}(C1) = 1 - \mathbf{Success}(C1)$$

$$0 \leq Importance(B_1) \leq 1$$

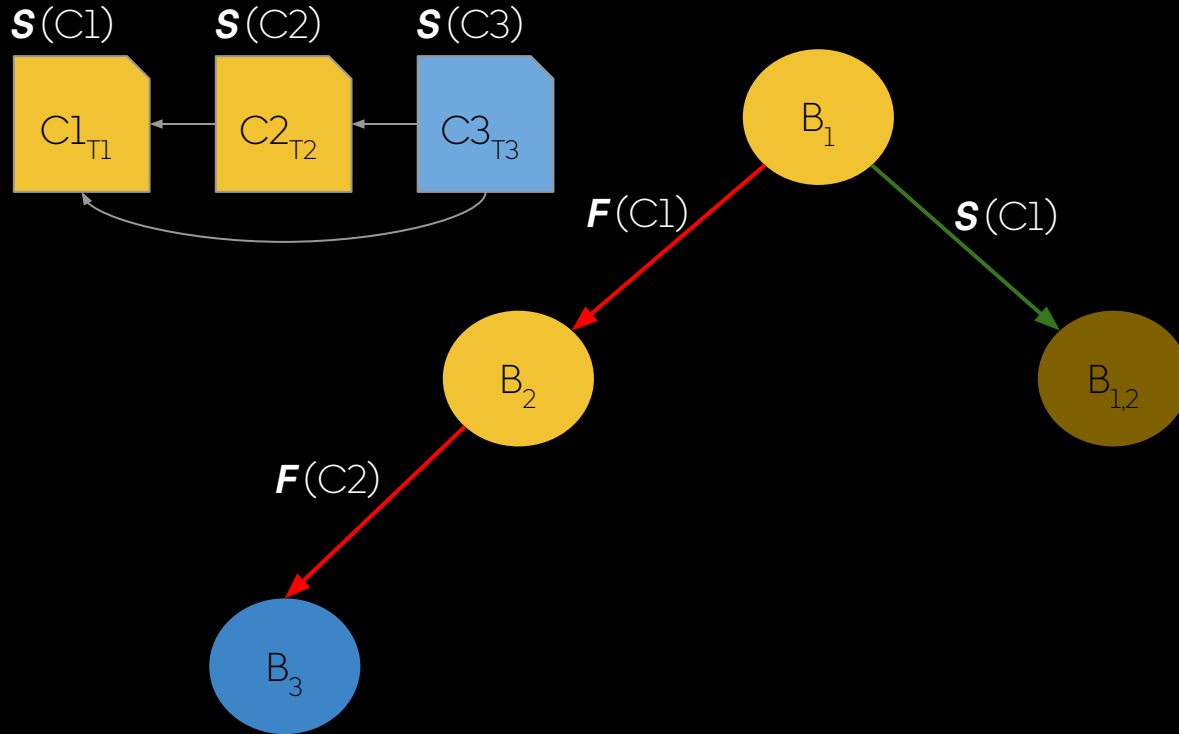
Probabilistic Speculation Tree (C2)



Probabilistic Speculation Tree (C2)

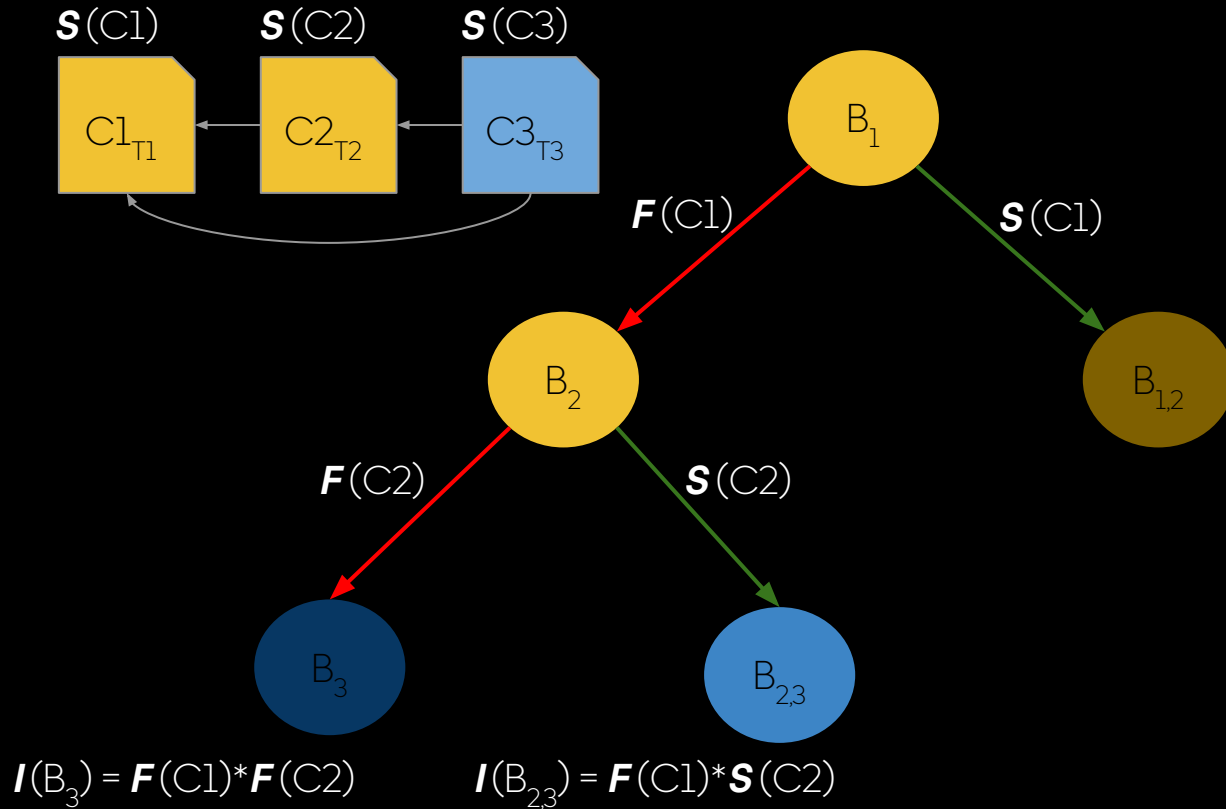


Probabilistic Speculation Tree (C3)

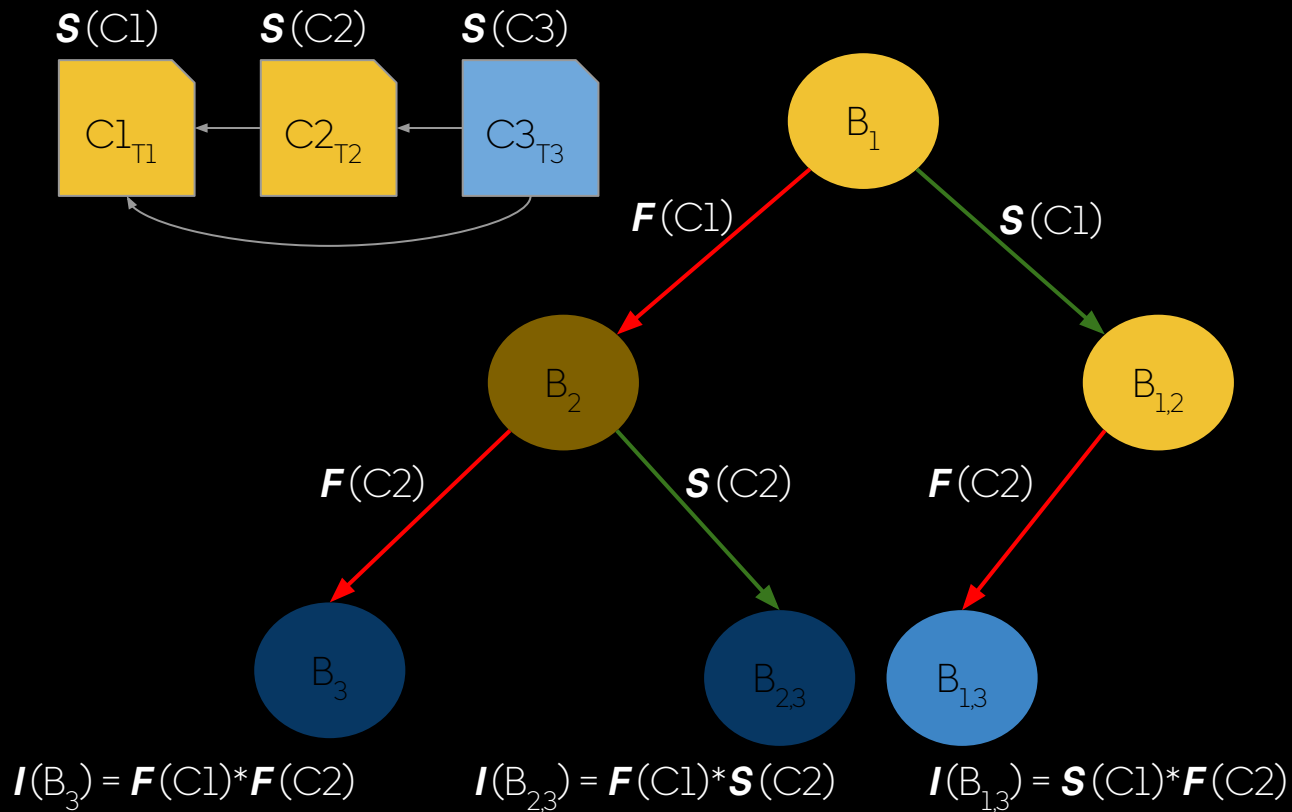


$$I(B_3) = F(C1) * F(C2)$$

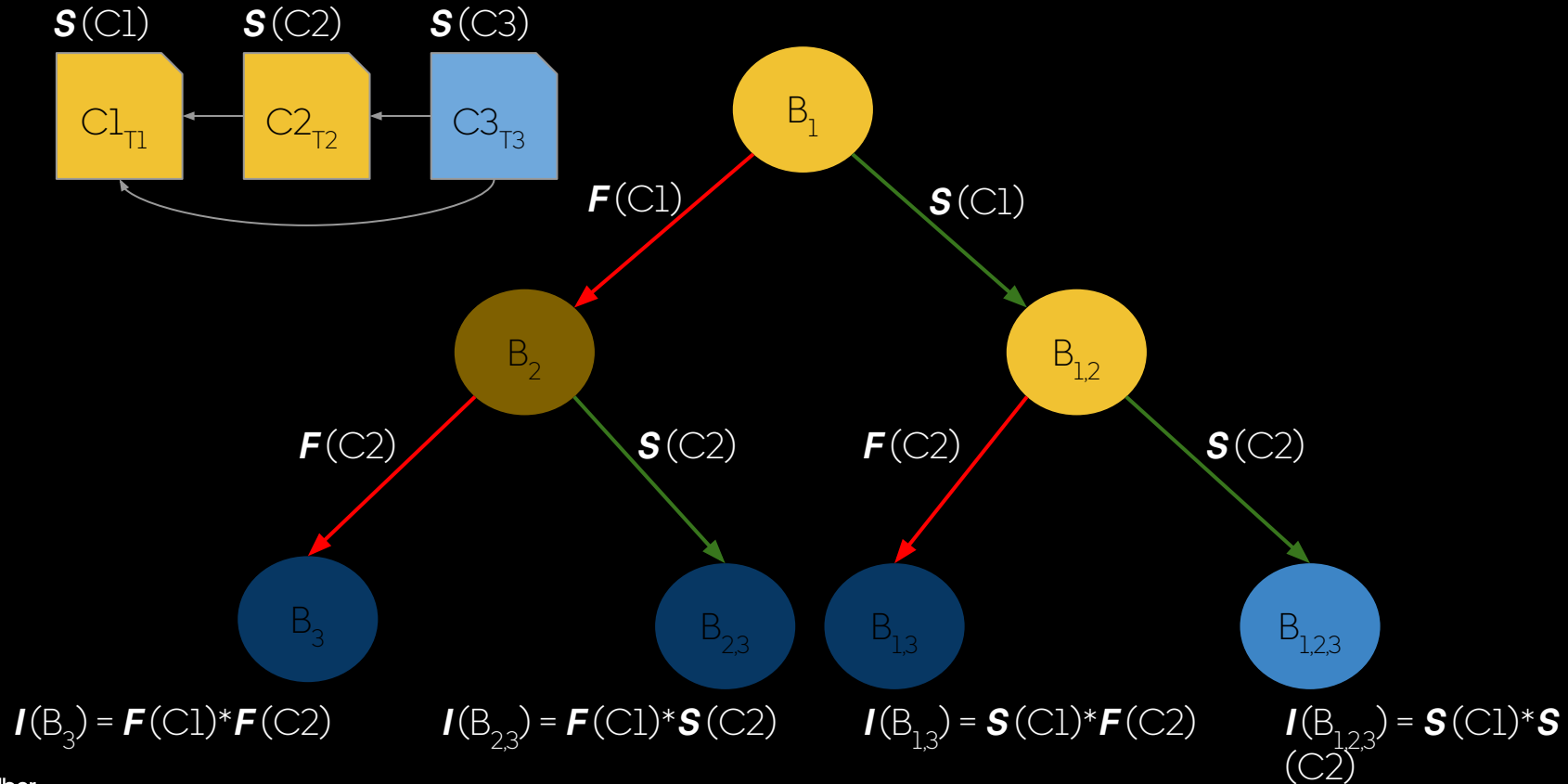
Probabilistic Speculation Tree (C3)



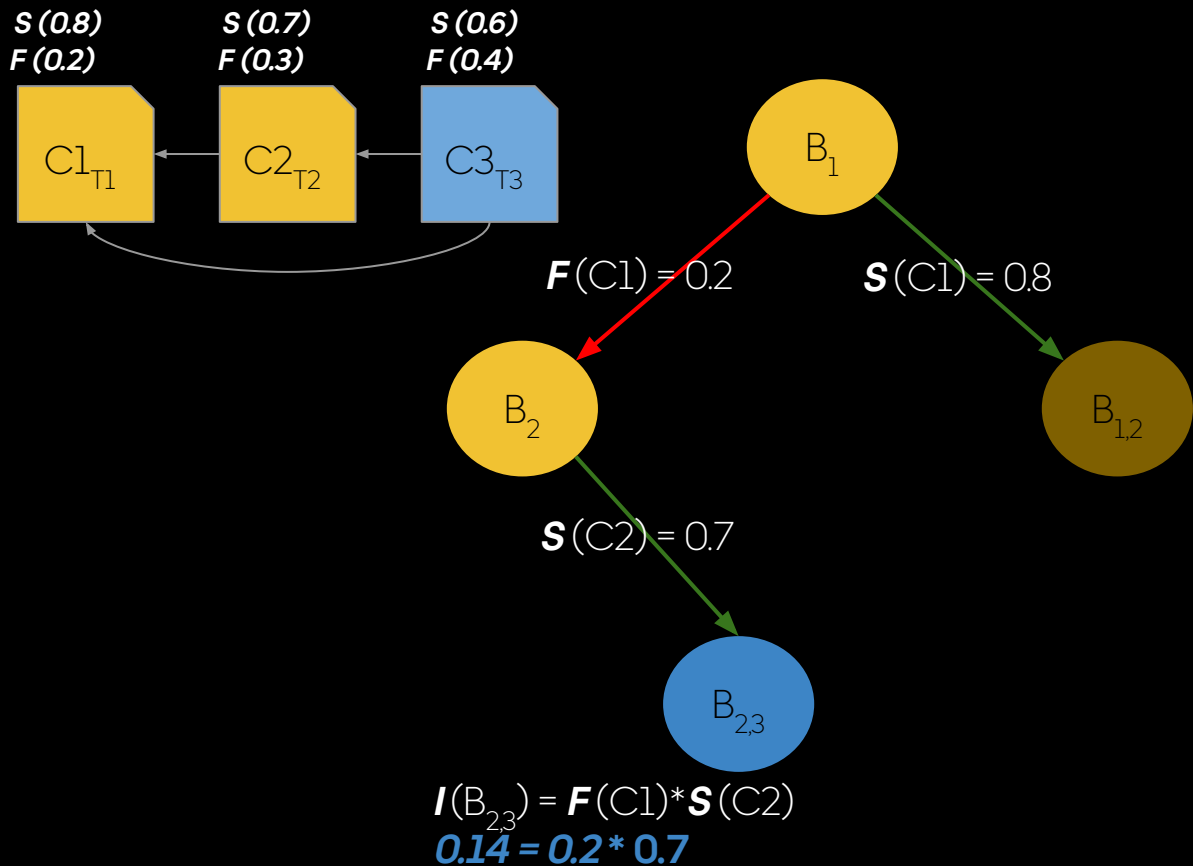
Probabilistic Speculation Tree (C3)



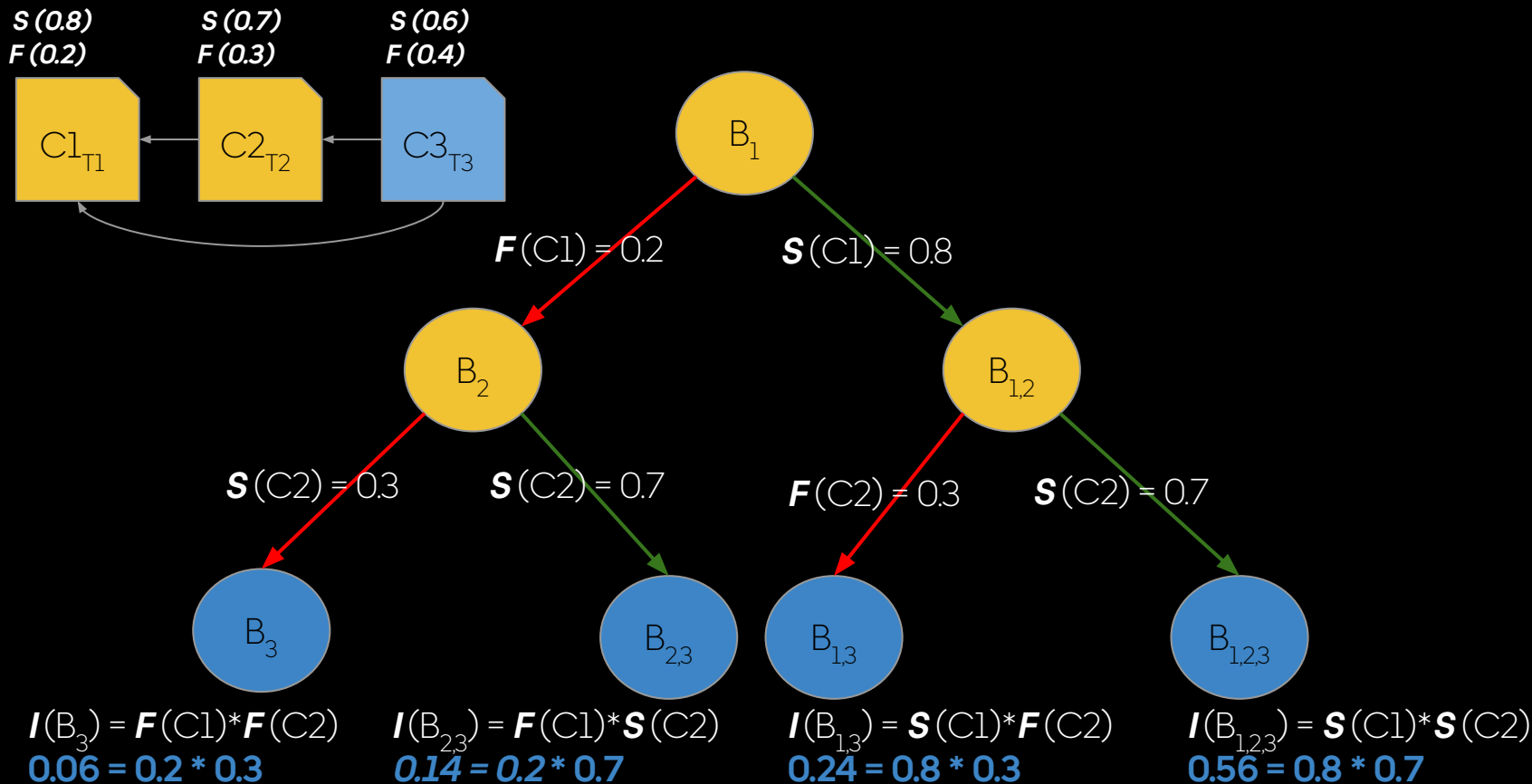
Probabilistic Speculation Tree (C3)



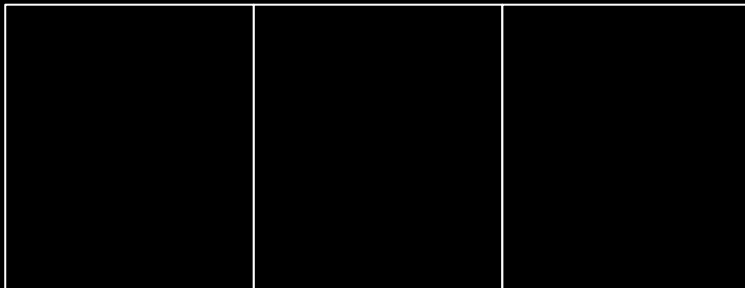
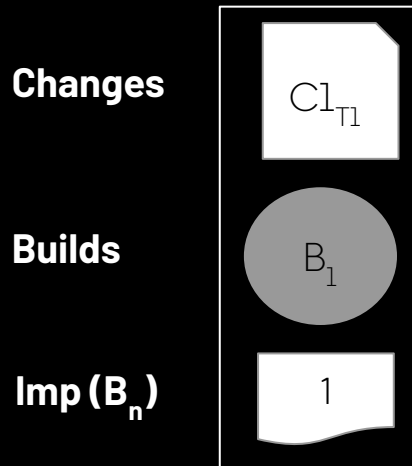
Probabilistic Speculation Tree: Scoring



Probabilistic Speculation Tree: Scoring



Build Prioritization & Scheduling

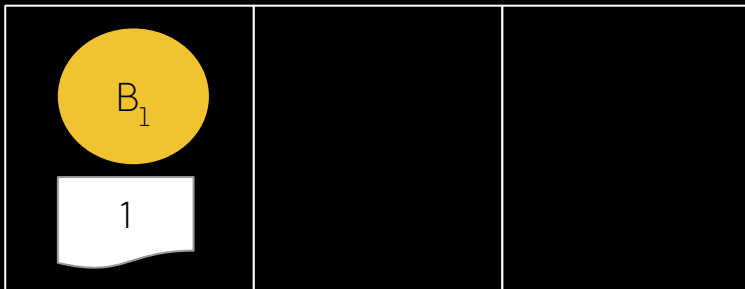
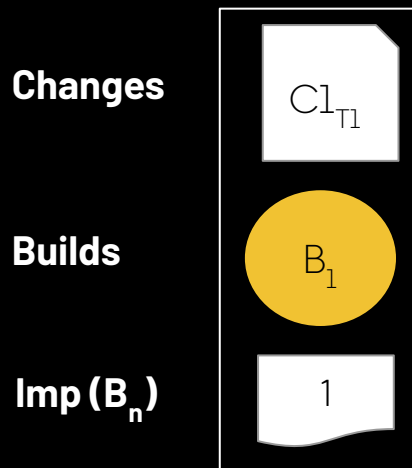


CI Resources: 3 builds at a time



Build to be scheduled

Build Prioritization & Scheduling

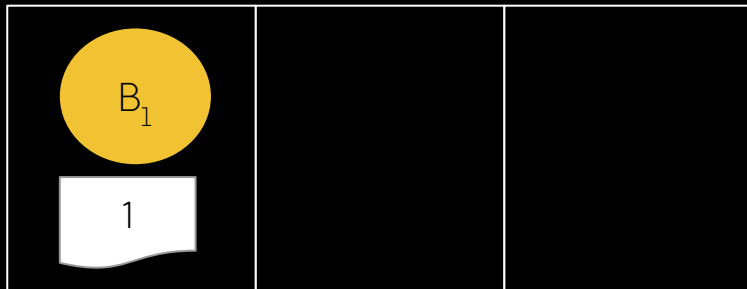
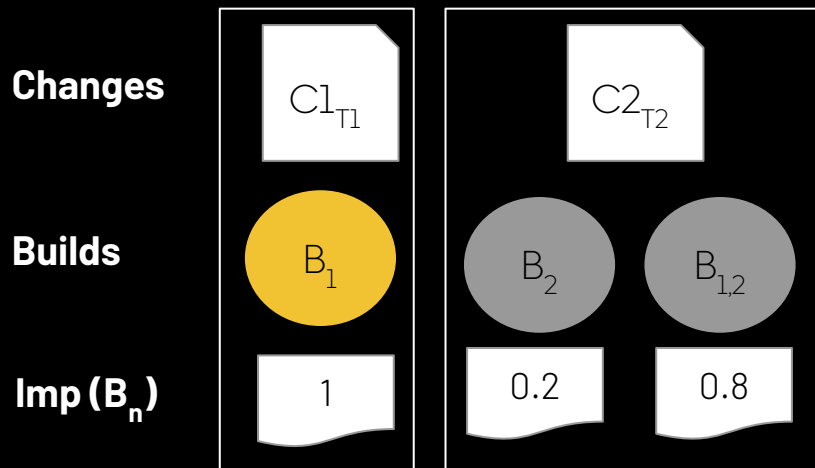


CI Resources: 3 builds at a time



Build scheduled

Build Prioritization & Scheduling



CI Resources: 3 builds at a time

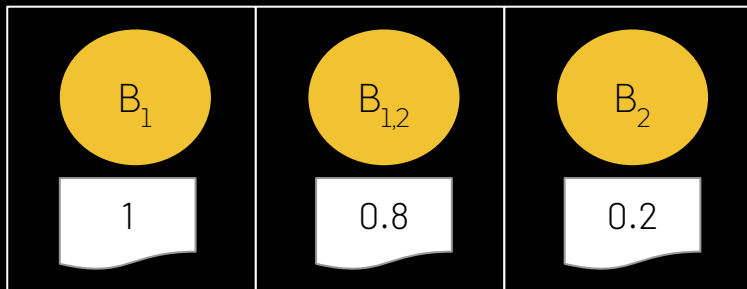
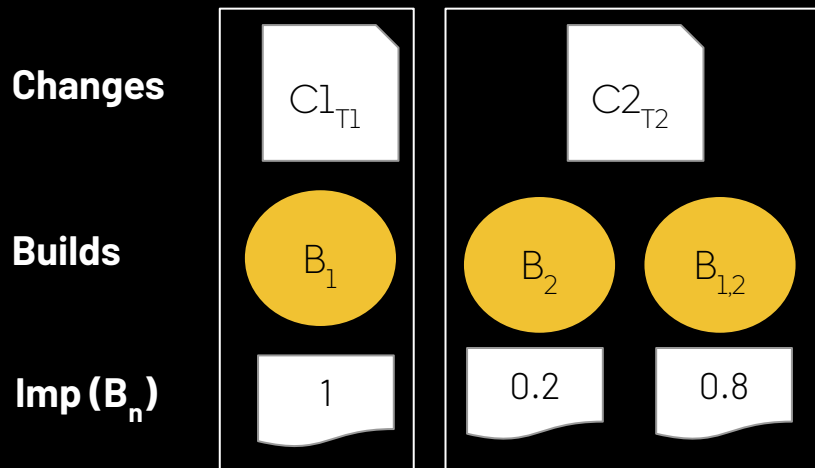


Build to be scheduled



Build scheduled

Build Prioritization & Scheduling

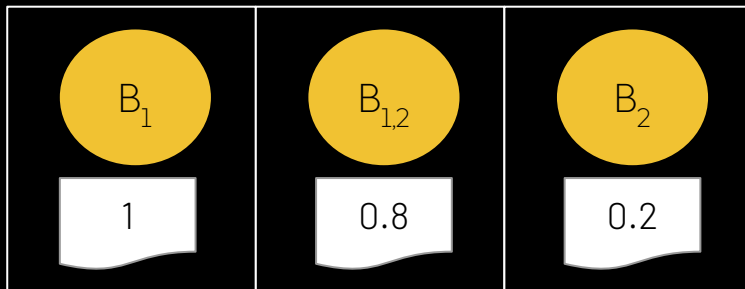
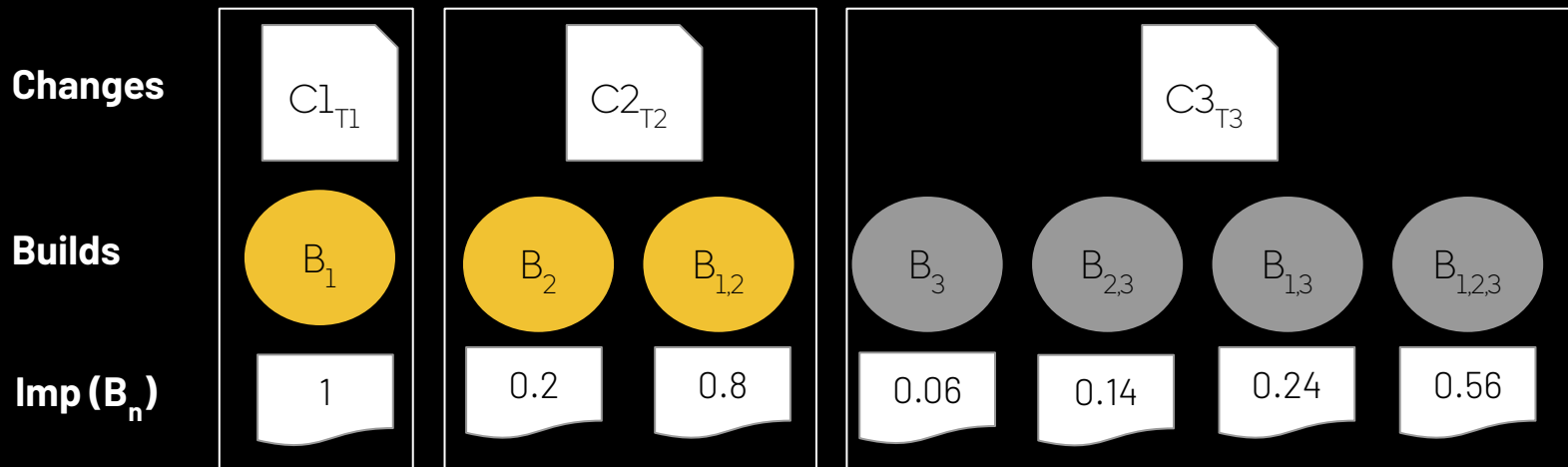


CI Resources: 3 builds at a time

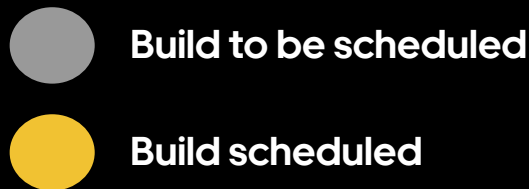


Build scheduled

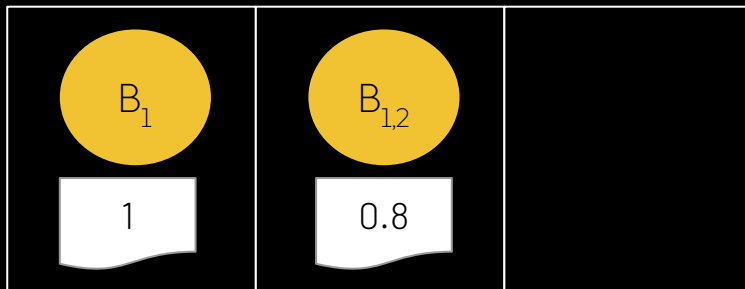
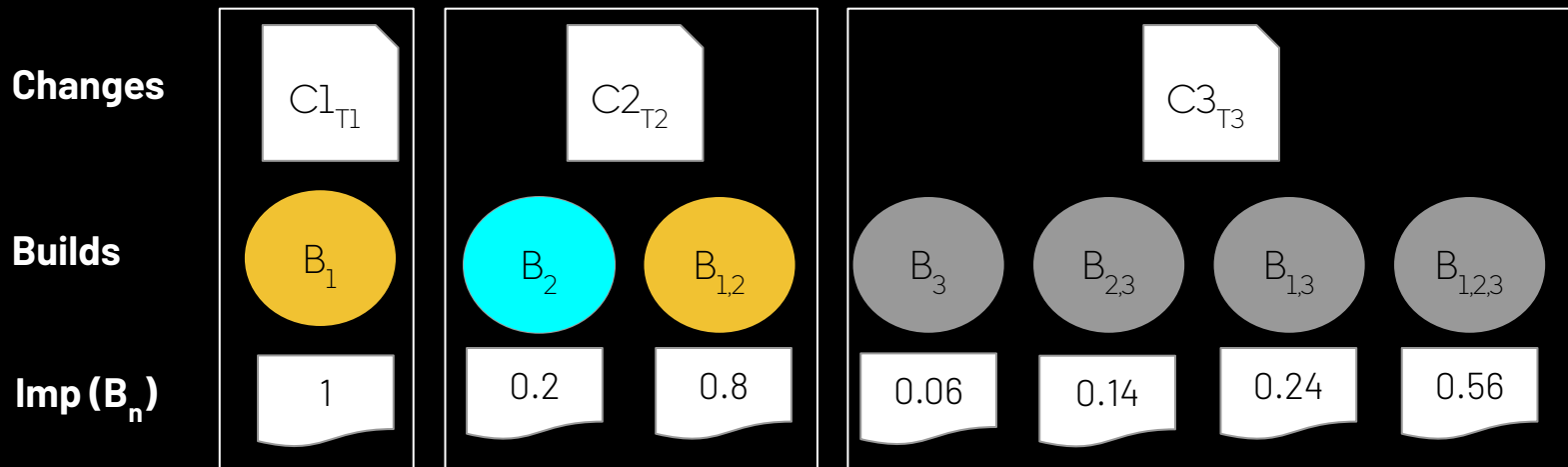
Build Prioritization & Scheduling



CI Resources: 3 builds at a time



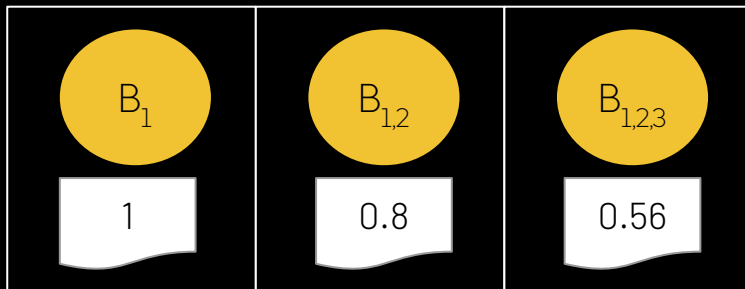
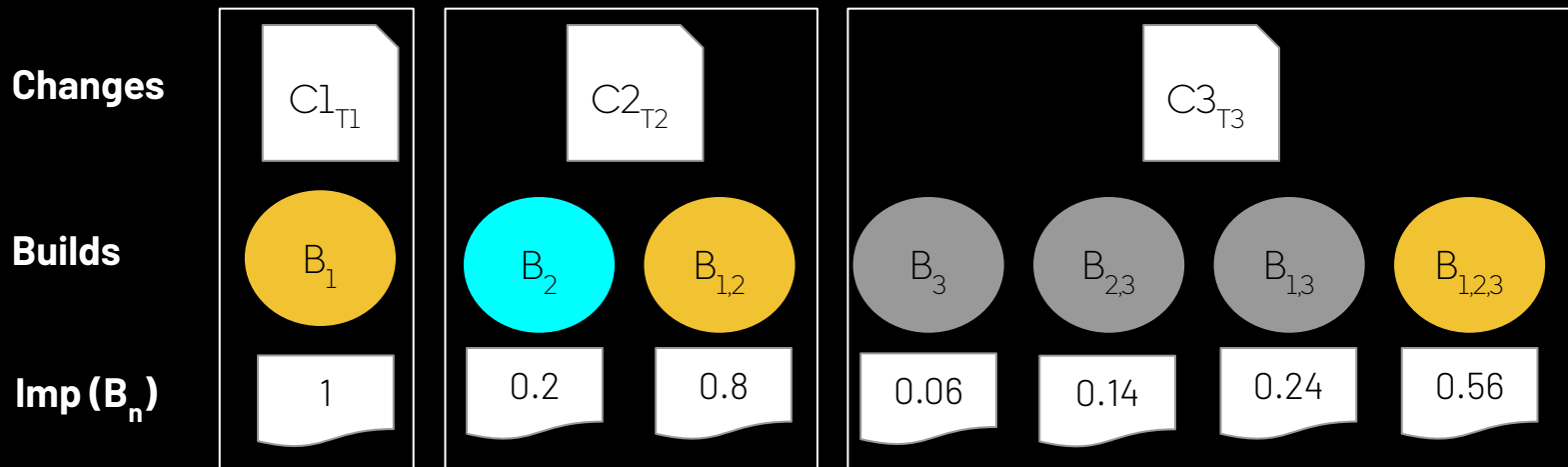
Build Prioritization & Scheduling



CI Resources: 3 builds at a time

-  Build to be scheduled
-  Build scheduled
-  Build aborted: lower importance

Build Prioritization & Scheduling



CI Resources: 3 builds at a time



Build to be scheduled

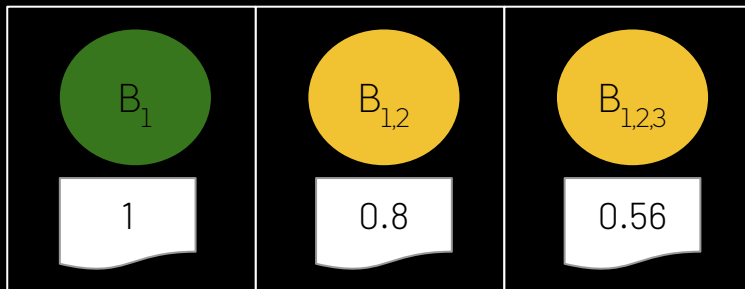
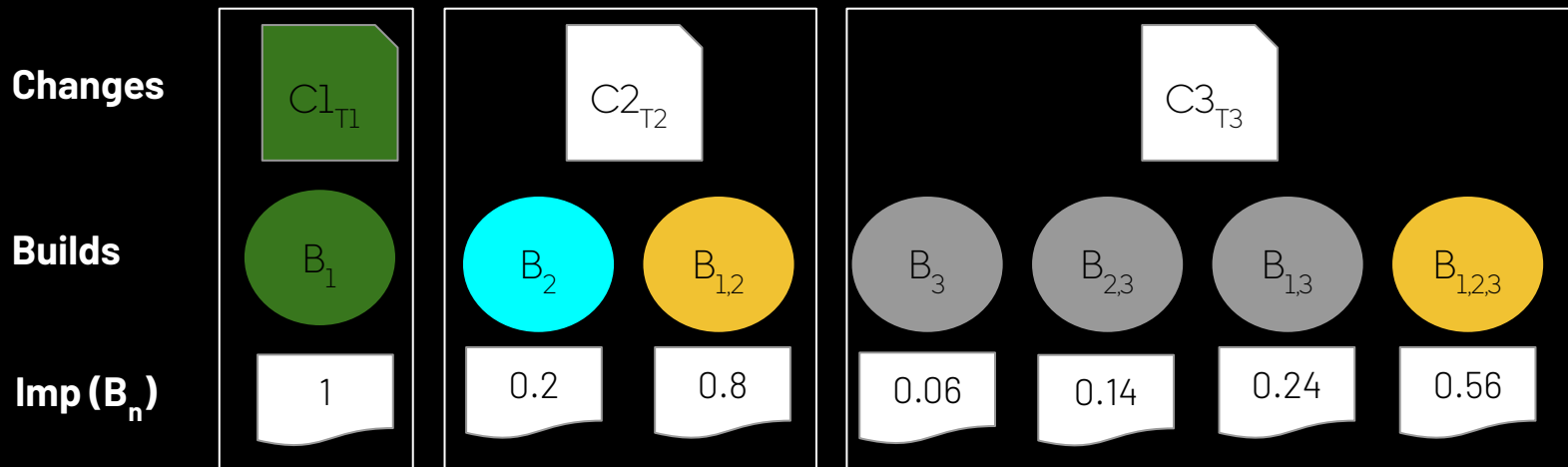


Build scheduled



Build aborted: lower importance

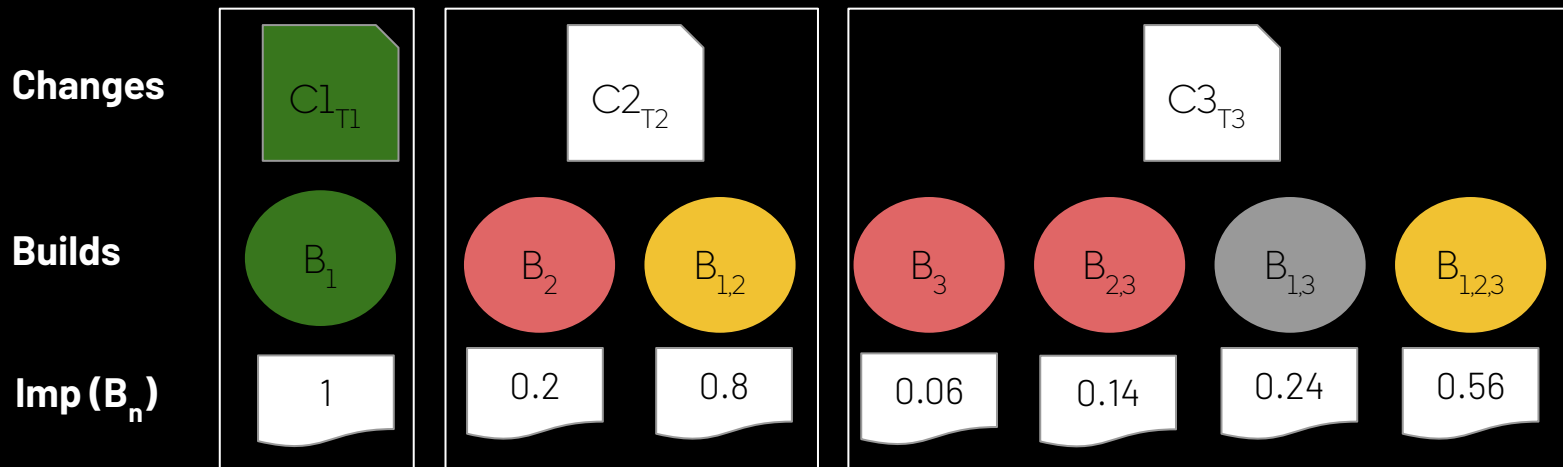
Build Prioritization & Scheduling



CI Resources: 3 builds at a time

-  Build to be scheduled
-  Build scheduled
-  Build aborted: lower importance
-  Build passed

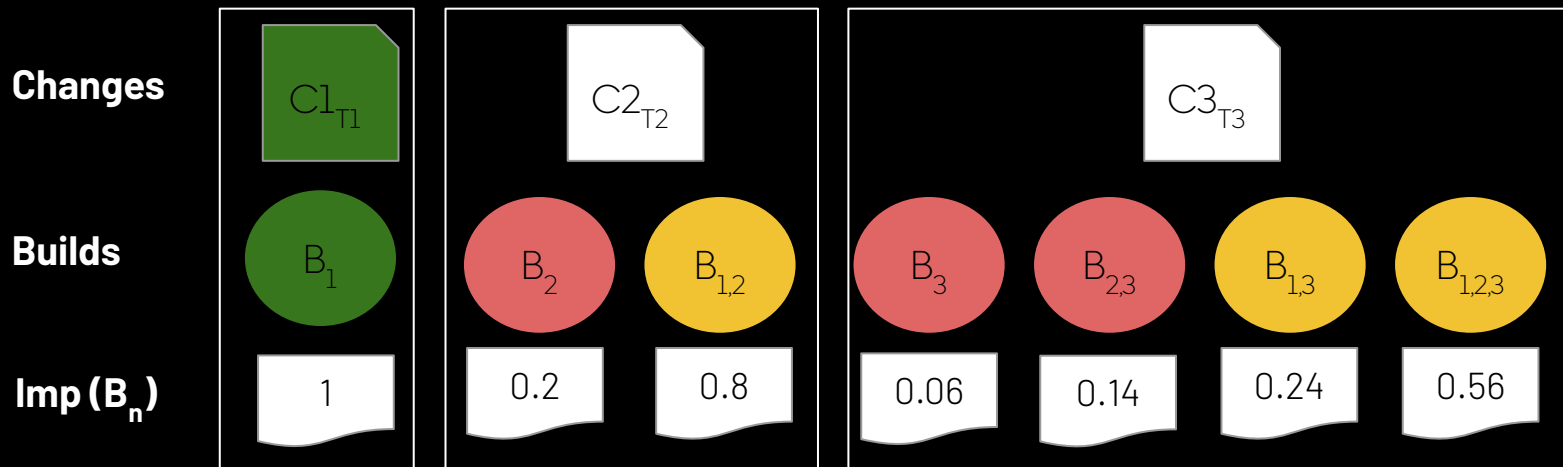
Build Prioritization & Scheduling



CI Resources: 3 builds at a time

-  Build to be scheduled
-  Build scheduled
-  Build passed
-  Build invalidated

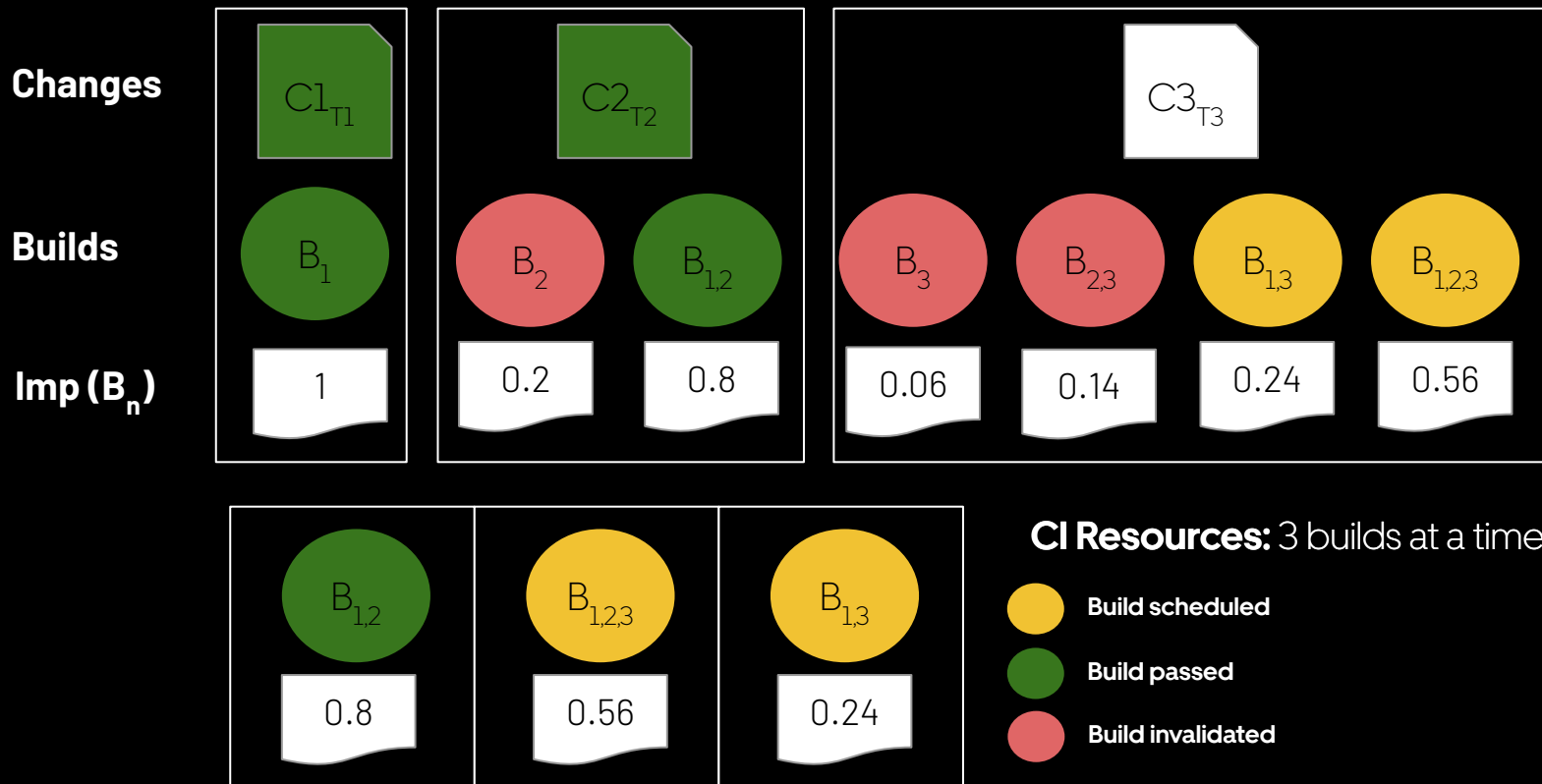
Build Prioritization & Scheduling



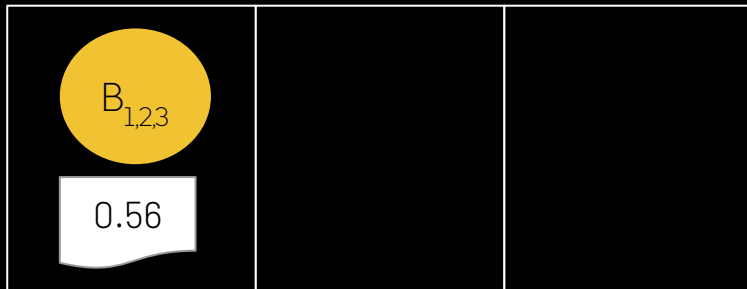
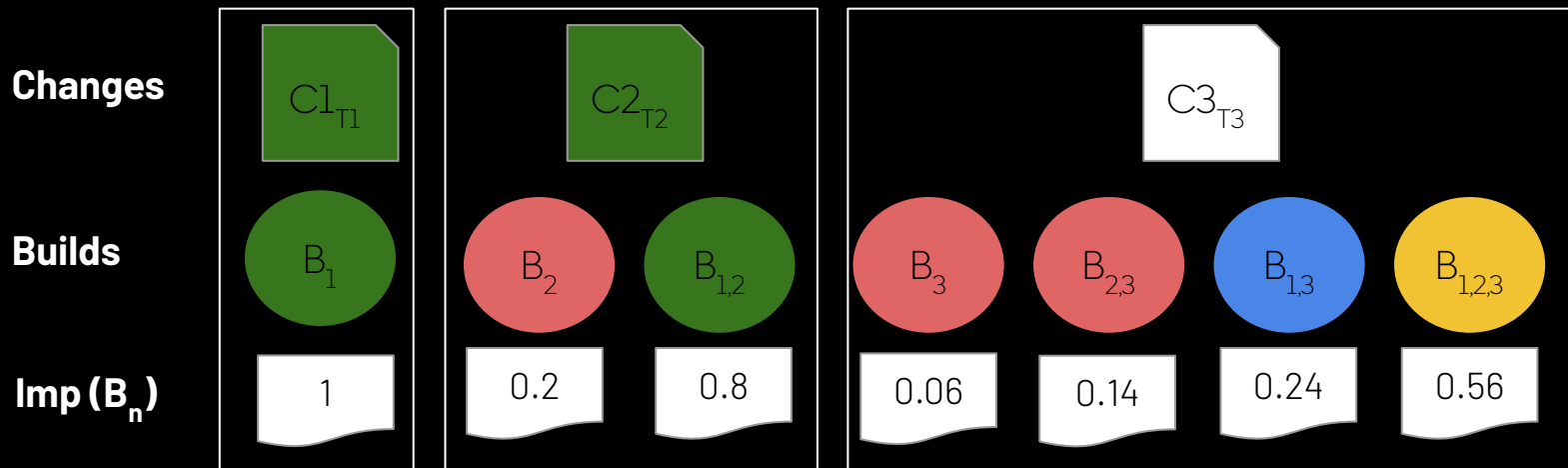
CI Resources: 3 builds at a time

- Build scheduled
- Build passed
- Build invalidated

Build Prioritization & Scheduling



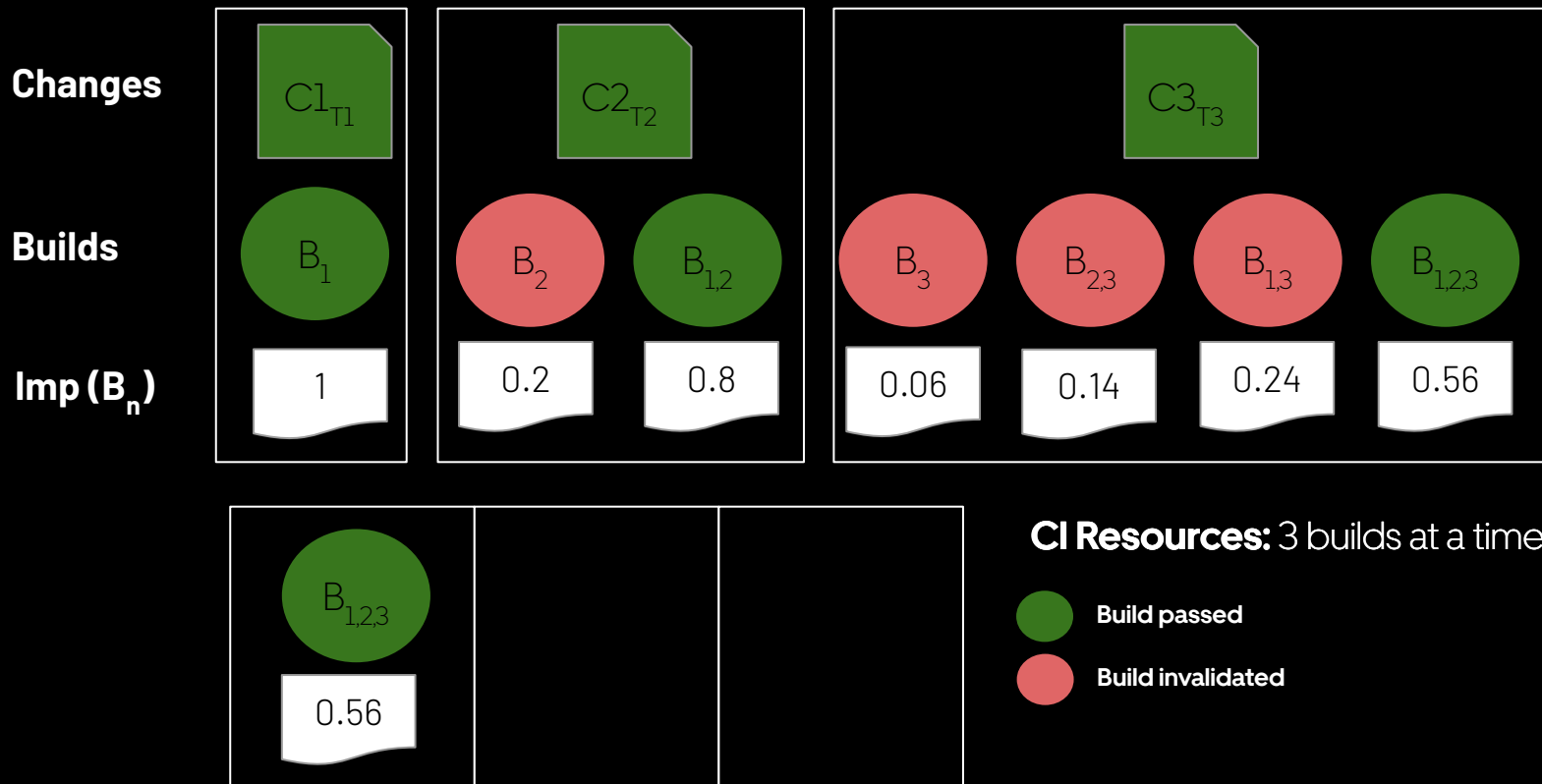
Build Prioritization & Scheduling



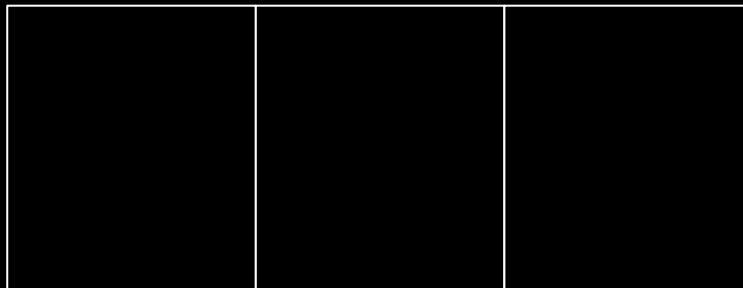
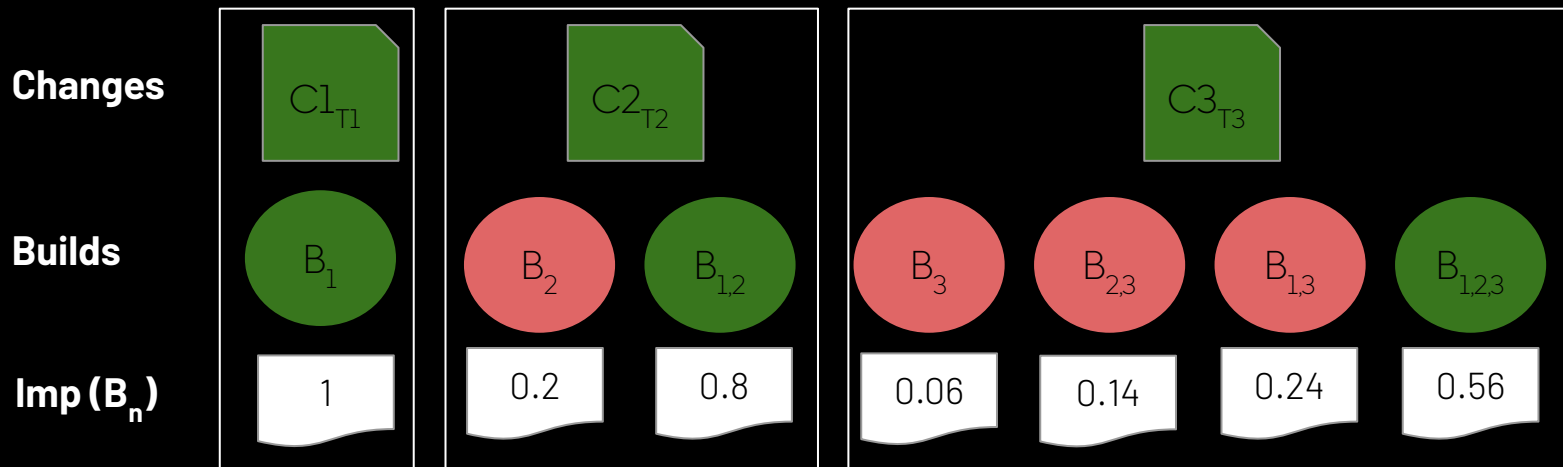
CI Resources: 3 builds at a time

- Build scheduled
- Build passed
- Build aborted: assumptions invalidated
- Build invalidated

Build Prioritization & Scheduling



Build Prioritization & Scheduling



CI Resources: 3 builds at a time

- Build passed
- Build invalidated

Optimization-4: Fast track smaller changes

Optimizing throughput by allowing **changes that build faster** to move ahead



Dynamic Flow

Departure from strict FIFO constraints to prevent pipeline stalls.



Out-of-Order

Small, fast changes leapfrog larger, time-intensive changes.

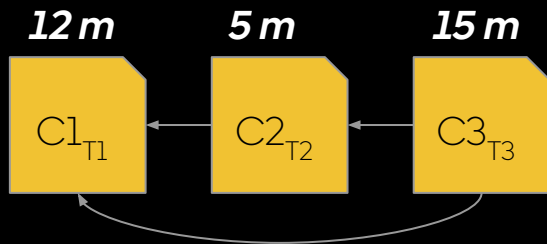


Safe Acceleration

Maximizes throughput without compromising safety protocols.

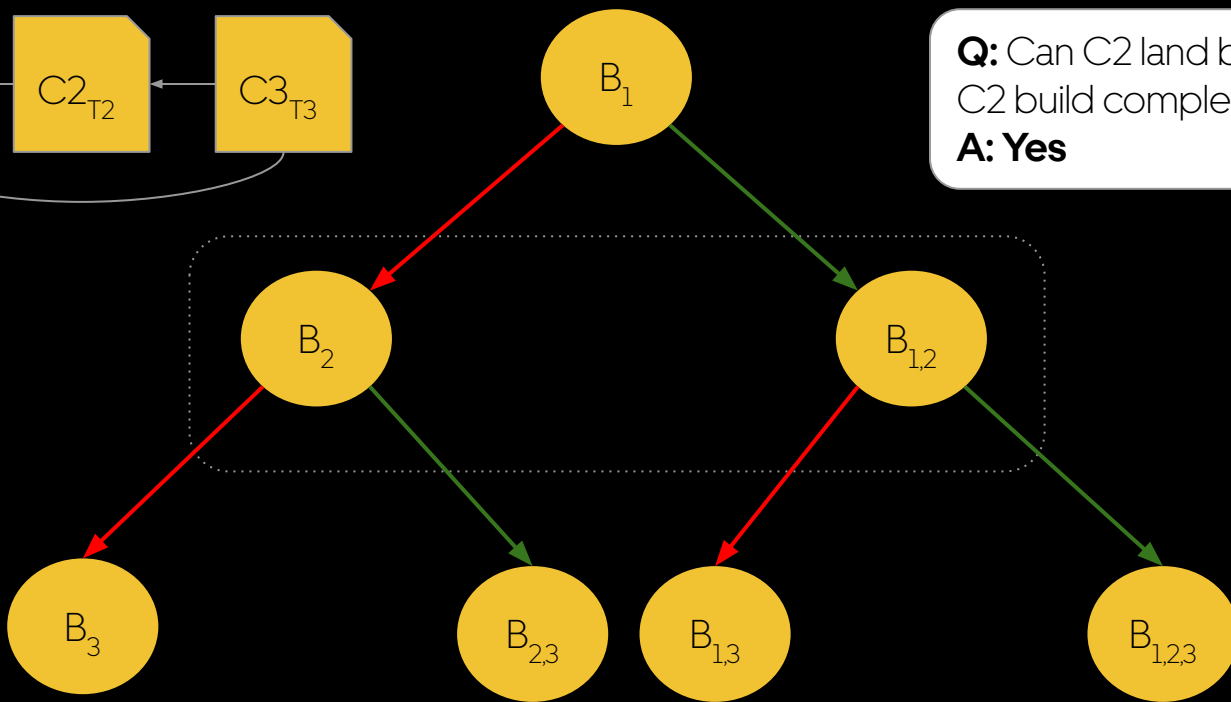
Let's look at an example to understand?

Optimization-4: Fast track smaller changes

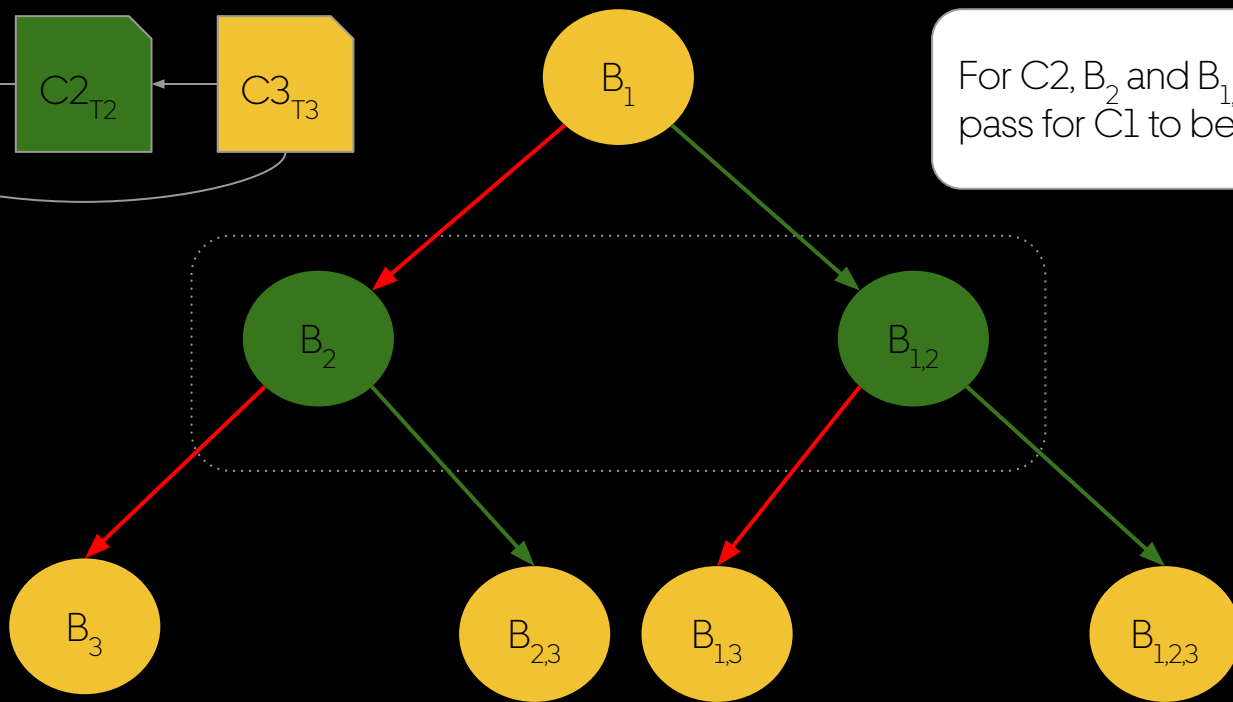
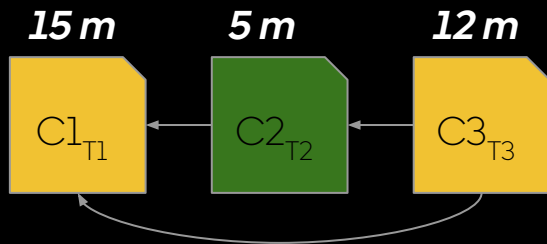


Q: Can C2 land before C1? If C2 build completes faster.

A: Yes



Optimization-4: Fast track smaller changes

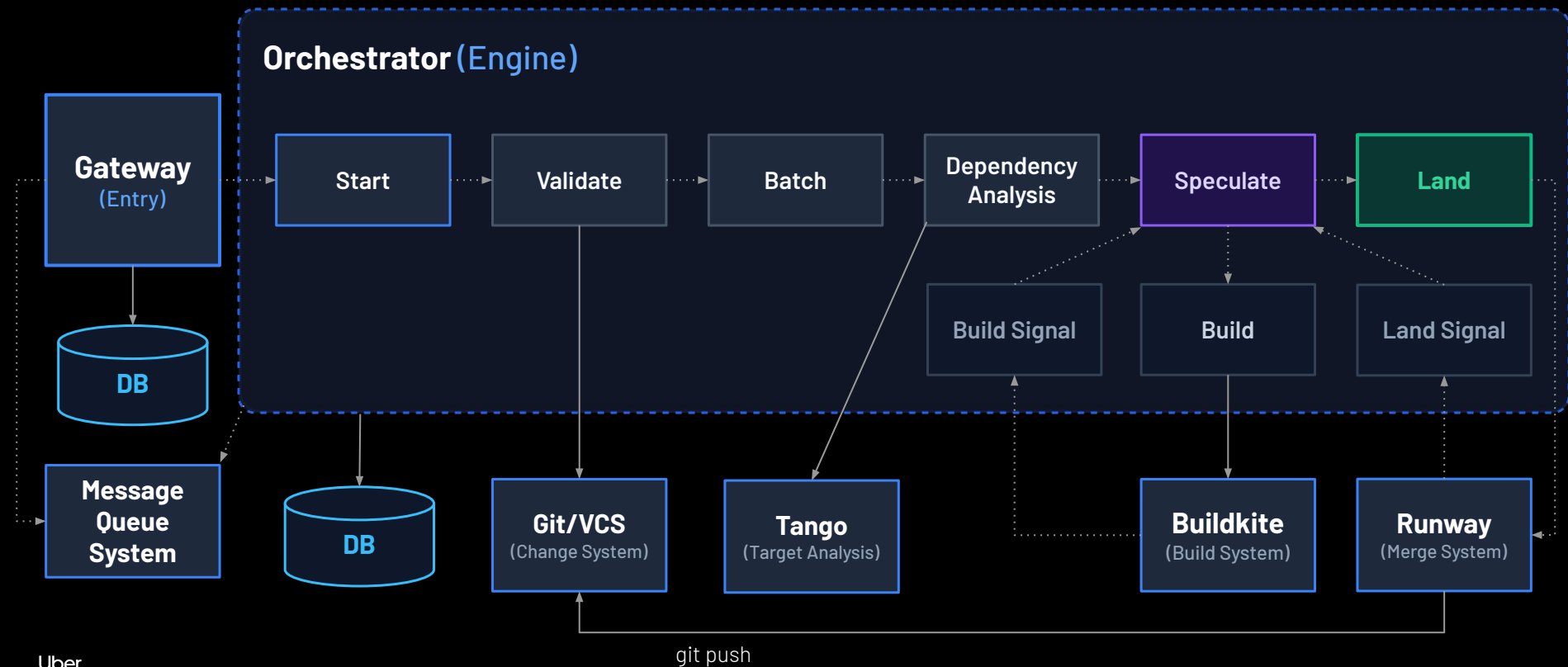


For $C2$, B_2 and $B_{1,2}$ both must pass for $C1$ to be bypassed

Architecture, Evolution & Outlook

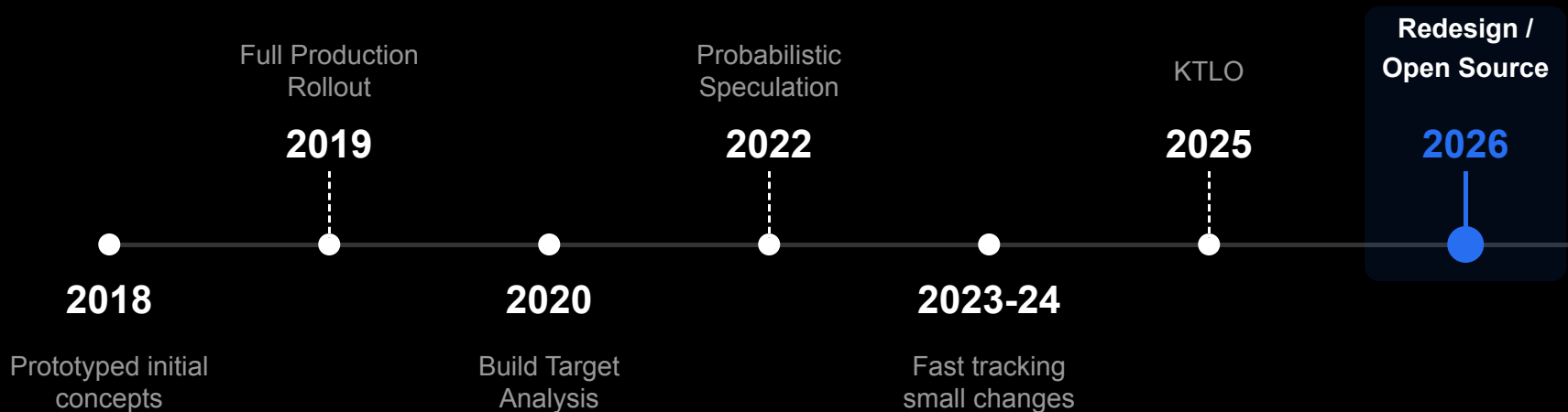
Design, evolution, scale, and what lies ahead

SubmitQueue Architecture



Evolution of SubmitQueue

How the system evolved with scale and complexity



Scale & Performance

~7K

Daily Commits (Monorepos)

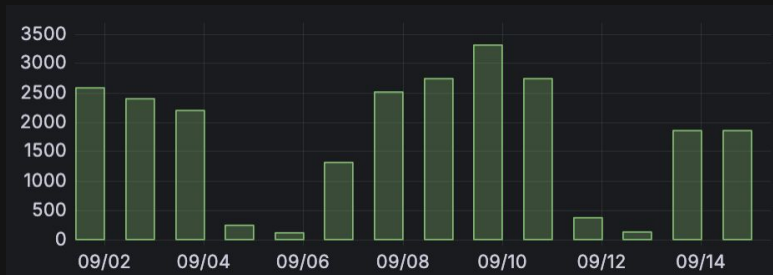
~5K+

Active Daily Contributors

Go Monorepo: The Biggest

As Uber's largest monorepo, Go handles the bulk of high-velocity development.

- 3,000+ engineers contributing daily
- ~4k commits merged per day
- ~250 PRs lands/hr (peak)



Limitations

Where the system still falls short



Throughput & Merge Efficiency

High Conflicts & Reduced Throughput

High conflicts can lead to lower throughput - large pull requests and frequent overlapping changes significantly slow down queue processing.



Incident Response

No Fast Drain

Traffic spikes and incidents cause backlogs with no mechanism for rapid queue clearing.

Future Plan

How we're addressing these challenges



Parallelism

Conflict Relaxation

Increase speculation parallelism by dynamically relaxing conflict constraints.



Reliability

Trunk Validation

Validate broken trunks post-merge using *Stovepipe* as a side effect of conflict relaxation.



Efficiency

Batching

Reduce speculation search space through intelligent request batching.

Open Source

Explore the codebase and contribution guidelines



GITHUB REPOSITORY

Scan to view and explore

Access the codebase, review guidelines, and
contribute directly on GitHub.

<https://github.com/uber/submitqueue>



Questions & Discussion

Scan to connect on LinkedIn — happy to continue the conversation



Preetam Dwivedi
@behinddwalls



Manjari Akella
@manjari-akella-b7a0a791